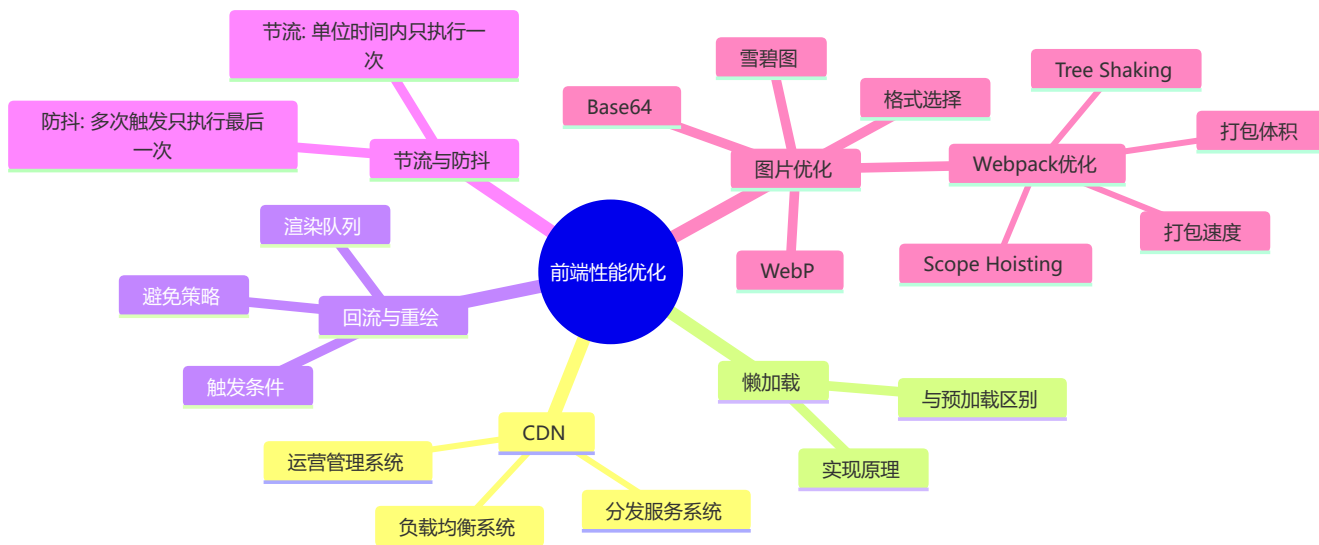


⚡ 前端性能优化知识详解版（含 Mermaid 图解）

🎯 面试星级：★★★★☆ | 建议用时：2 天

前端性能优化核心知识全面梳理，涵盖 Core Web Vitals、资源加载、渲染优化、构建优化、前沿架构等核心考点

🧠 知识脑图



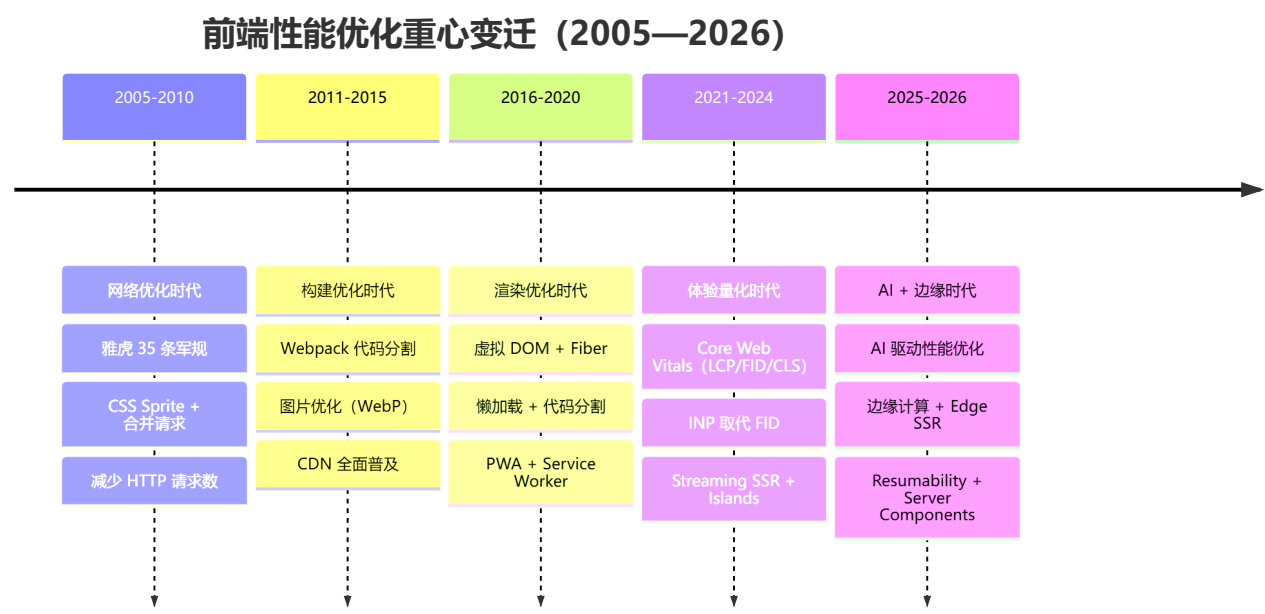
📋 目录

- 📖 [性能优化技术演进史](#)
- 🌐 [一、CDN](#)
- 🚀 [二、懒加载](#)
- 🔄 [三、回流与重绘](#)
- 🎯 [四、节流与防抖](#)
- 🖼️ [五、图片优化](#)
- 📦 [六、Webpack优化](#)
- 📊 [七、Core Web Vitals](#)
- 🚀 [八、资源加载优化](#)
- 🎨 [九、渲染性能优化](#)
- 📈 [十、监测与分析](#)
- 🌐 [十一、前沿性能技术](#)
- 🚀 [十二、2025/2026 性能优化新趋势](#)

📈 性能优化技术演进史

性能优化的重心随 Web 应用形态的演变而不断迁移。

四个时代的性能重心



关键性能指标演进

时期	核心指标	关注点	工具
2005-2010	页面加载时间	网络请求数、资源大小	YSlow、PageSpeed
2011-2015	首屏时间	构建产物、缓存策略	Webpack Analyzer
2016-2020	TTI / FMP	交互可用性、渲染阻塞	Lighthouse 初版
2021-2024	LCP / FID / CLS	用户体验量化	Lighthouse + CrUX
2025-2026	LCP / INP / CLS	交互响应 + 加载 + 视觉稳定	Lighthouse AI + RUM

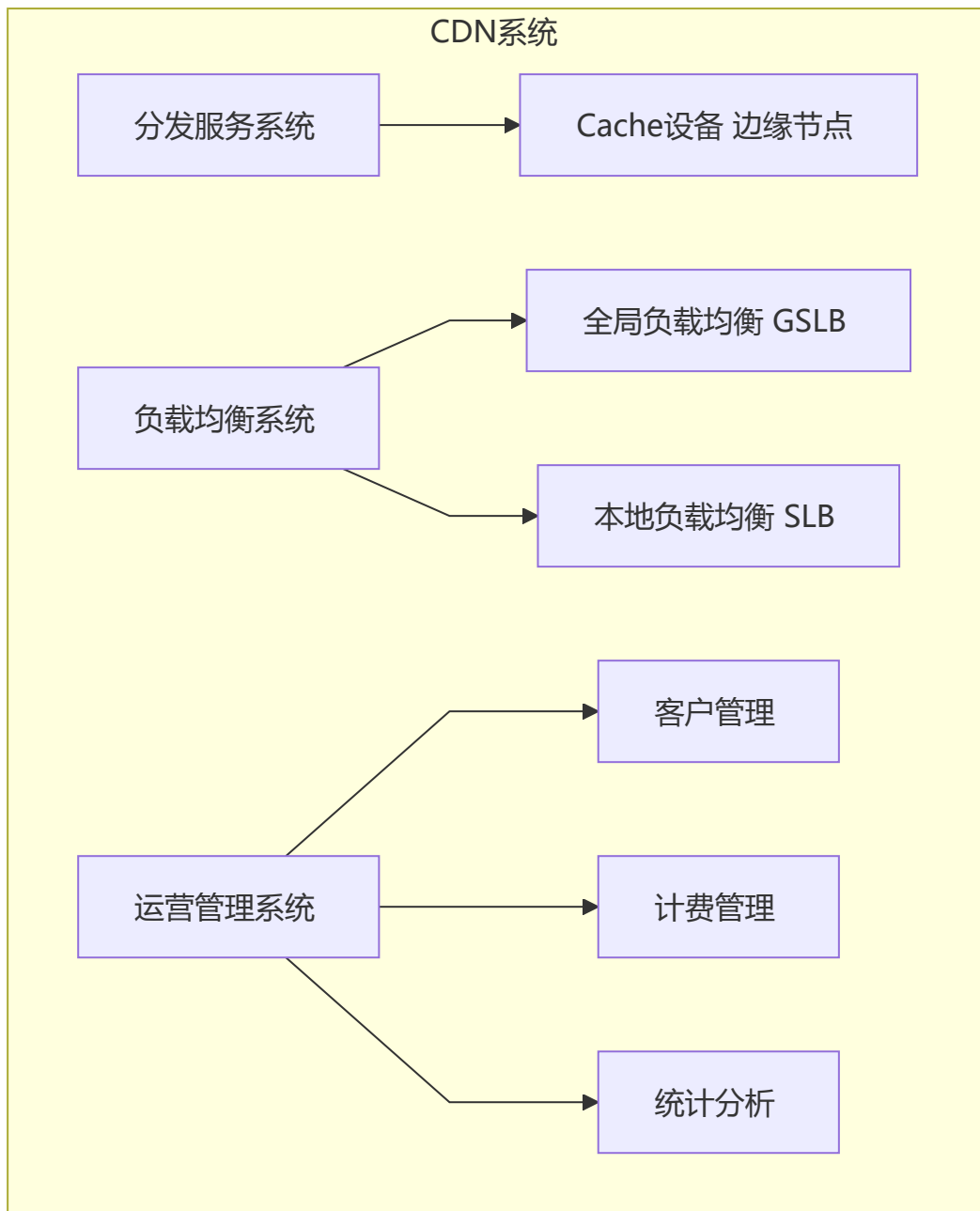
性能优化范式对比

范式	代表技术	核心理念	适用场景
传统优化	雪碧图、合并请求、压缩	减少传输	HTTP/1.1 时代
构建优化	Tree Shaking、代码分割	减少体积	Webpack 时代
渲染优化	Virtual DOM、Incremental DOM	减少成本	SPA 时代
流式优化	Streaming SSR、Islands	尽快展示	同构时代
预测优化	Speculation Rules、Prefetch	提前加载	现代浏览器
AI 优化	AI 分析、自适应加载	智能决策	2025+

1 CDN 的概念

💡 **要点：** CDN（内容分发网络）通过将内容缓存到离用户最近的节点来加速交付，由分发服务系统、负载均衡系统、运营管理系统三大子系统组成。

CDN（Content Delivery Network，**内容分发网络**）是指一种通过互联网互相连接的电脑网络系统，利用最靠近每位用户的服务器，更快、更可靠地将音乐、图片、视频、应用程序及其他文件发送给用户，来提供高性能、可扩展性及低成本的网络内容传递给用户。



典型的CDN系统由下面三个部分组成：

- **分发服务系统：**最基本的工作单元就是Cache设备，cache（边缘cache）负责直接响应最终用户的访问请求，把缓存在本地的内容快速地提供给用户。同时cache还负责与源站点进行内容同步，把更新的内容以及本地没有的内容从源站点获取并保存在本地。

- **负载均衡系统**：主要功能是负责对所有发起服务请求的用户进行访问调度，确定提供给用户的最终实际访问地址。两级调度体系分为全局负载均衡（GSLB）和本地负载均衡（SLB）。**全局负载均衡**主要根据用户就近性原则，通过对每个服务节点进行"最优"判断，确定向用户提供服务的cache的物理位置。**本地负载均衡**主要负责节点内部的设备负载均衡
- **运营管理系统**：运营管理系统分为运营管理和网络管理子系统，负责处理业务层面的与外界系统交互所必须的收集、整理、交付工作。

2 CDN 的作用

要点：CDN 在性能上能降低用户访问延迟、减轻源服务器负载；在安全上能有效防御 DDoS 和 MITM 等网络攻击。

(1) 在性能方面，引入CDN的作用在于：

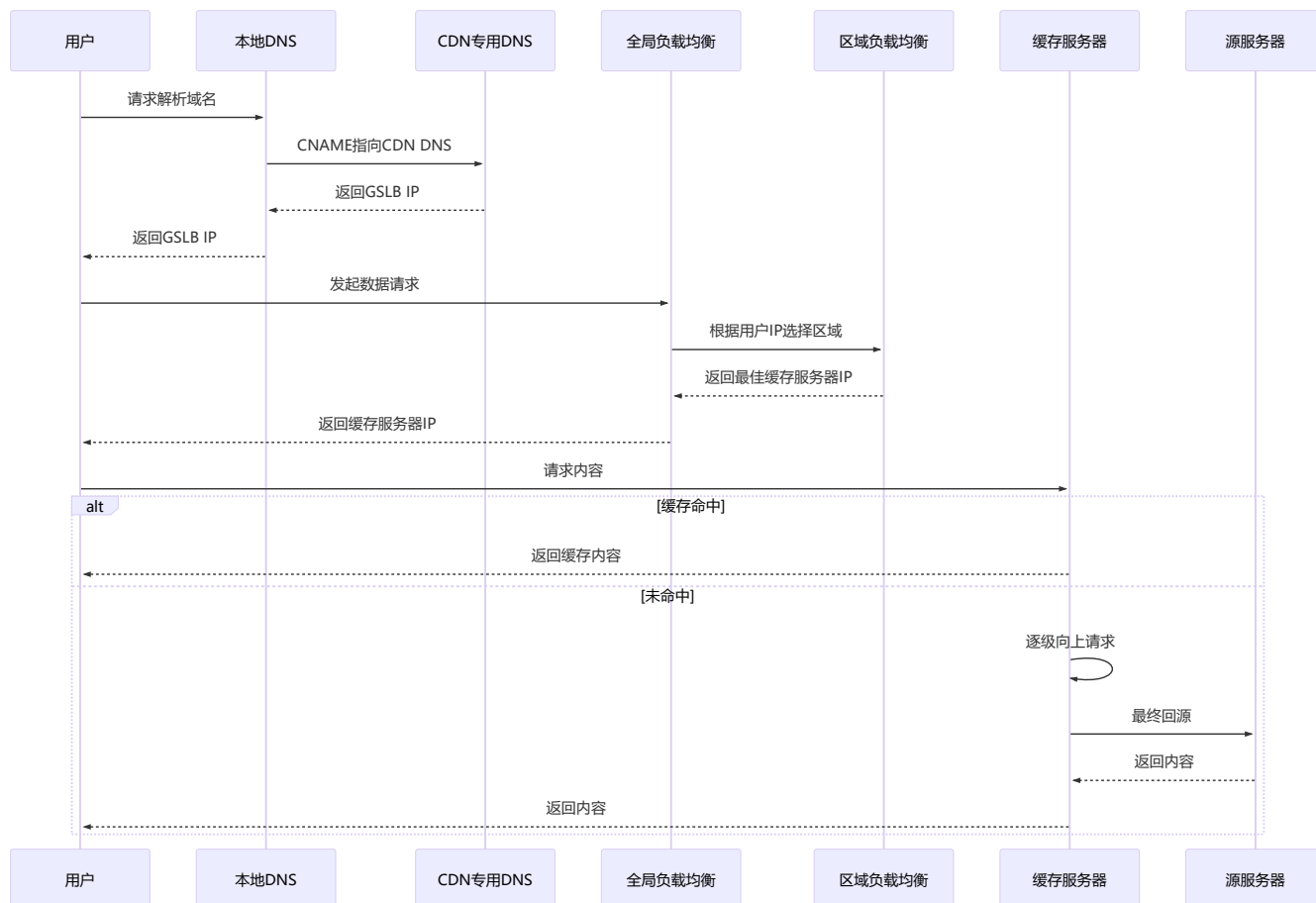
- 用户收到的内容来自最近的数据中心，延迟更低，内容加载更快
- 部分资源请求分配给了CDN，减少了服务器的负载

(2) 在安全方面，CDN有助于防御DDoS、MITM等网络攻击：

- 针对DDoS：通过监控分析异常流量，限制其请求频率
- 针对MITM：从源服务器到 CDN 节点到 ISP，全链路 HTTPS 通信

3 CDN 的原理

要点：CDN 通过 CNAME 解析将域名指向 CDN 专用 DNS，再经由全局负载均衡（GSLB）和区域负载均衡（SLB）选择最优缓存节点，实现就近访问。



CDN的工作原理：

(1) 用户未使用CDN缓存资源的过程：

1. 浏览器通过DNS对域名进行解析，依次得到此域名对应的IP地址
2. 浏览器根据得到的IP地址，向域名的服务主机发送数据请求
3. 服务器向浏览器返回响应数据

(2) 用户使用CDN缓存资源的过程：

1. 经过本地DNS系统的解析，发现该URL对应的是一个CDN专用的DNS服务器，DNS系统就会将域名解析权交给CNAME指向的CDN专用的DNS服务器。
2. CDN专用DNS服务器将CDN的全局负载均衡设备IP地址返回给用户
3. 用户向CDN的全局负载均衡设备发起数据请求
4. CDN的全局负载均衡设备根据用户的IP地址以及用户请求的内容URL，选择一台用户所属区域的区域负载均衡设备
5. 区域负载均衡设备选择一台合适的缓存服务器来提供服务，将该缓存服务器的IP地址返回给全局负载均衡设备
6. 全局负载均衡设备把服务器的IP地址返回给用户
7. 用户向该缓存服务器发起请求，缓存服务器响应用户的请求

 **补充说明：**CNAME（Canonical Name，别名记录）在域名解析中，实际上解析出来的可能是域名对应的IP地址，也可能是该域名的CNAME记录，需要再根据CNAME查找对应的IP地址。CDN正是利用CNAME将域名解析权交给CDN专用DNS，从而实现请求的智能调度。

4 CDN 的使用场景

 **要点：**CDN 适用于第三方库引用加速、静态资源缓存分发、以及直播流媒体传送等场景。

- **使用第三方的CDN服务：**如果想要开源一些项目，可以使用第三方的CDN服务
- **使用CDN进行静态资源的缓存：**将自己网站的静态资源放在CDN上，比如js、css、图片等。可以将整个项目放在CDN上，完成一键部署。
- **直播传送：**直播本质上是使用流媒体进行传送，CDN也是支持流媒体传送的

zz 二、懒加载

1 懒加载的概念

 **要点：**懒加载（延迟加载/按需加载）指在长网页中延迟加载图片等资源，仅在需要时才加载，是常用的网页性能优化方式。

懒加载也叫做延迟加载、按需加载，指的是在长网页中延迟加载图片数据，是一种较好的网页性能优化的方式。

2 懒加载的特点

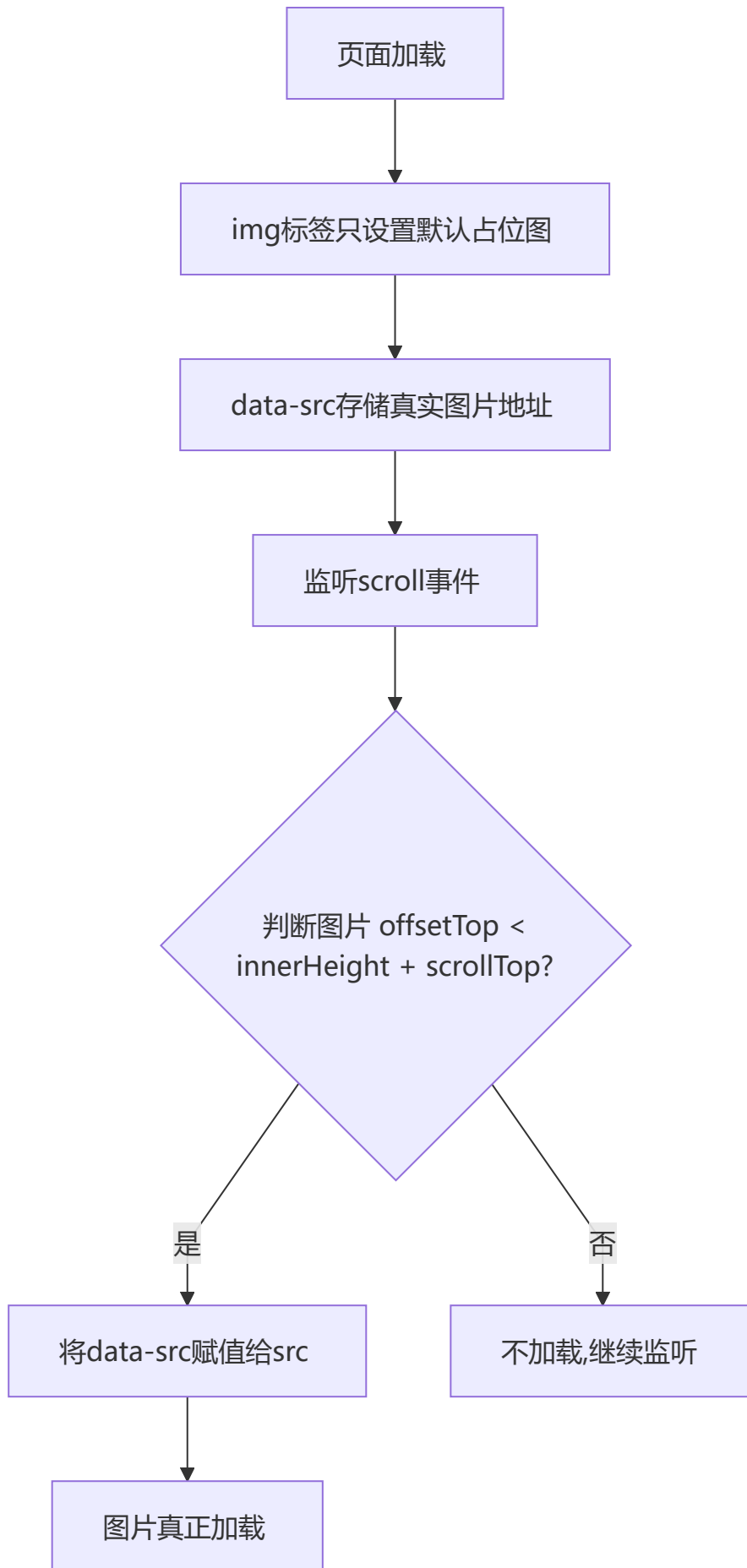
 **要点：**懒加载可减少无用资源加载、提升用户体验、防止过多图片阻塞其他资源加载。

- **减少无用资源的加载：**明显减少了服务器的压力和流量
- **提升用户体验：**使用懒加载能大大地提高用户体验

- 防止加载过多图片而影响其他资源文件的加载

3 懒加载的实现原理

💡 **要点：**通过 data-src 存储真实图片地址，监听 scroll 事件判断元素 offsetTop 是否进入视口，进入时再将 data-src 赋值给 src 触发加载。



📄 **说明：**传统懒加载基于 scroll 事件监听 + offsetTop 判断，性能较差。现代推荐使用 **Intersection Observer API**，性能更好且无需频繁计算滚动位置。

代码实现（传统方式）：

```
<div class="container">
  
  
  
</div>
<script>
var imgs = document.querySelectorAll('img');
function lazyLoad(){
  var scrollTop = document.body.scrollTop || document.documentElement.scrollTop;
  var winHeight= window.innerHeight;
  for(var i=0;i < imgs.length;i++){
    if(imgs[i].offsetTop < scrollTop + winHeight ){
      imgs[i].src = imgs[i].getAttribute('data-src');
    }
  }
}
window.onscroll = lazyLoad;
</script>
```

⚠ **注意：**传统 scroll 监听方案会频繁触发计算，实际生产环境中建议使用 `IntersectionObserver` 或 `loading="lazy"` 属性（现代浏览器原生支持）。

4 懒加载与预加载的区别

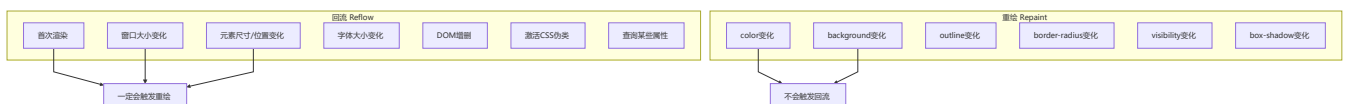
💡 **要点：**懒加载是延迟按需加载以减轻首屏压力；预加载是提前加载到缓存以提升后续访问速度，两者策略相反。

- **懒加载：**当用户需要访问时，再去加载，提高首屏加载速度，减少服务器压力
- **预加载：**将所需的资源提前请求加载到本地，后面需要时直接从缓存取资源

🔄 三、回流与重绘

1 回流与重绘的概念及触发条件

💡 **要点：**回流（Reflow）是元素尺寸/结构变化导致重新布局，重绘（Repaint）是仅样式变化不影响布局。回流必触发重绘，重绘不一定引发回流。



◆ (1) 回流

当渲染树中部分或者全部元素的尺寸、结构或者属性发生变化时，浏览器会重新渲染部分或者全部文档的过程就称为**回流**。

下面这些操作会导致回流：

- 页面的首次渲染
- 浏览器的窗口大小发生变化
- 元素的内容发生变化
- 元素的尺寸或者位置发生变化
- 元素的字体大小发生变化
- 激活CSS伪类
- 查询某些属性或者调用某些方法
- 添加或者删除可见的DOM元素

在触发回流（重排）的时候，由于浏览器渲染页面是基于流式布局的，所以当触发回流时，会导致周围的DOM元素重新排列，它的影响范围有两种：

- 全局范围：从根节点开始，对整个渲染树进行重新布局
- 局部范围：对渲染树的某部分或者一个渲染对象进行重新布局

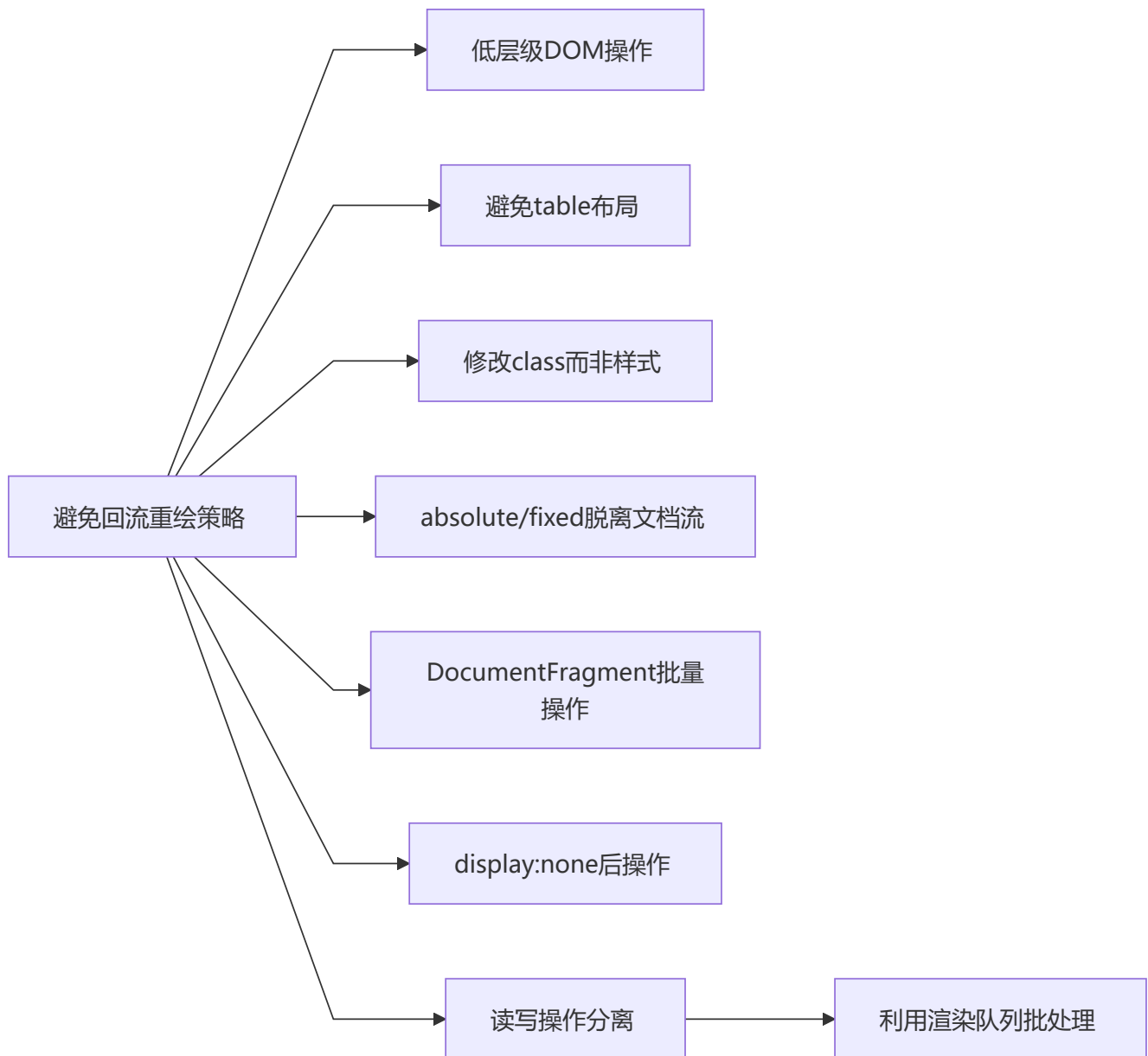
◆ (2) 重绘

当页面中某些元素的样式发生变化，但是不会影响其在文档流中的位置时，浏览器就会对元素进行重新绘制，这个过程就是**重绘**。

⚠ **注意：**当触发回流时，**一定会触发重绘**，但是重绘**不一定会**引发回流。因此在性能优化中，应优先减少回流操作，其影响范围远大于重绘。

2 如何避免回流与重绘？

💡 **要点：**通过低层级 DOM 操作、class 批量修改、脱离文档流、DocumentFragment 批量操作、读写分离利用渲染队列等方式避免回流与重绘。



减少回流与重绘的措施：

- 操作DOM时，尽量在低层级的DOM节点进行操作
- 不要使用 `table` 布局，一个小的改动可能会使整个 `table` 进行重新布局
- 使用CSS的表达式
- 不要频繁操作元素的样式，对于静态页面，可以修改类名，而不是样式。
- 使用`absolute`或者`fixed`，使元素脱离文档流
- 避免频繁操作DOM，可以创建一个文档片段 `documentFragment`
- 将元素先设置 `display: none`，操作结束后再把它显示出来
- 将DOM的多个读操作（或者写操作）放在一起

⚠ **注意（渲染队列）**：浏览器维护一个**渲染队列**，当连续修改样式时，会将这些操作推入队列暂不执行，等批量处理或强制读取布局属性时才会触发回流。因此应避免交替执行读写操作——写操作后立即读操作会强制清空队列、多次回流。最佳实践是将写操作集中，读操作集中，分离执行。

3 如何优化动画？

💡 要点：将动画元素 position 设为 absolute/fixed 使其脱离文档流，这样回流不会影响其他元素布局，配合 will-change 和 transform 可进一步提升性能。

将动画的 position 属性设置为 absolute 或者 fixed，将动画脱离文档流，这样他的回流就不会影响到页面了。

4 DocumentFragment 是什么？

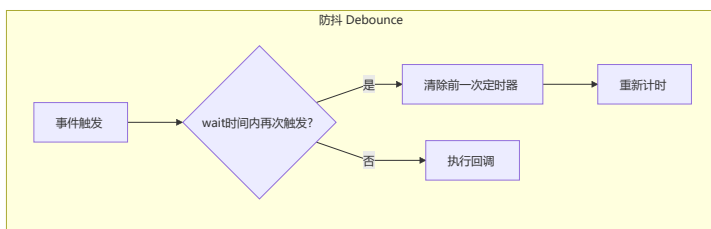
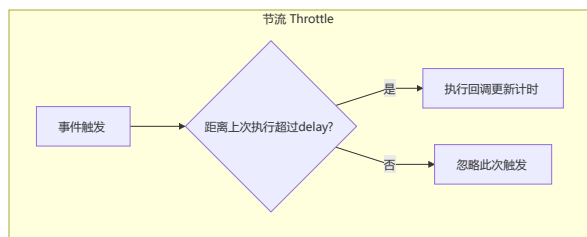
💡 要点：DocumentFragment 是轻量级文档片段，操作它不会触发 DOM 重渲染，插入到文档时合并所有子节点到真实 DOM，适用于批量 DOM 操作。

DocumentFragment，文档片段接口，一个没有父对象的最小文档对象。它被作为一个轻量版的 Document 使用，与 document 相比，最大的区别是 DocumentFragment 不是真实 DOM 树的一部分，它的变化不会触发 DOM 树的重新渲染。当我们把一个 DocumentFragment 节点插入文档树时，插入的不是 DocumentFragment 自身，而是它的所有子孙节点。

四、节流与防抖

1 对节流与防抖的理解

💡 要点：防抖（Debounce）连续触发只执行最后一次，适用于提交按钮、搜索联想；节流（Throttle）单位时间内只执行一次，适用于拖拽、resize、滚动事件。



防抖函数的应用场景：

- 按钮提交场景：防止多次提交按钮，只执行最后提交的一次
- 服务端验证场景：表单验证需要服务端配合，只执行一段连续的输入事件的最后一次

节流函数的适用场景：

- 拖拽场景：固定时间内只执行一次，防止超高频次触发位置变动
- 缩放场景：监控浏览器resize
- 动画场景：避免短时间内多次触发动画引起性能问题

2 实现节流函数和防抖函数

💡 要点：防抖通过每次触发时清除并重置定时器实现；节流通通过时间戳差值判断或定时器锁实现，两者都返回包装函数以保留 this 和参数。

函数防抖的实现：

```
function debounce(fn, wait) {  
  var timer = null;
```

```

return function() {
  var context = this,
      args = [...arguments];
  if (timer) {
    clearTimeout(timer);
    timer = null;
  }
  timer = setTimeout(() => {
    fn.apply(context, args);
  }, wait);
};
}

```

函数节流的实现:

```

// 时间戳版
function throttle(fn, delay) {
  var preTime = Date.now();
  return function() {
    var context = this,
        args = [...arguments],
        nowTime = Date.now();
    if (nowTime - preTime >= delay) {
      preTime = Date.now();
      return fn.apply(context, args);
    }
  };
}

// 定时器版
function throttle (fun, wait){
  let timeout = null
  return function(){
    let context = this
    let args = [...arguments]
    if(!timeout){
      timeout = setTimeout(() => {
        fun.apply(context, args)
        timeout = null
      }, wait)
    }
  }
}

```

⚠ 注意：时间戳版节流会立即执行第一次，停止触发后不再执行；定时器版节流延迟执行第一次，停止触发后还会执行最后一次。可根据实际场景选择或组合使用两种方式。



五、图片优化

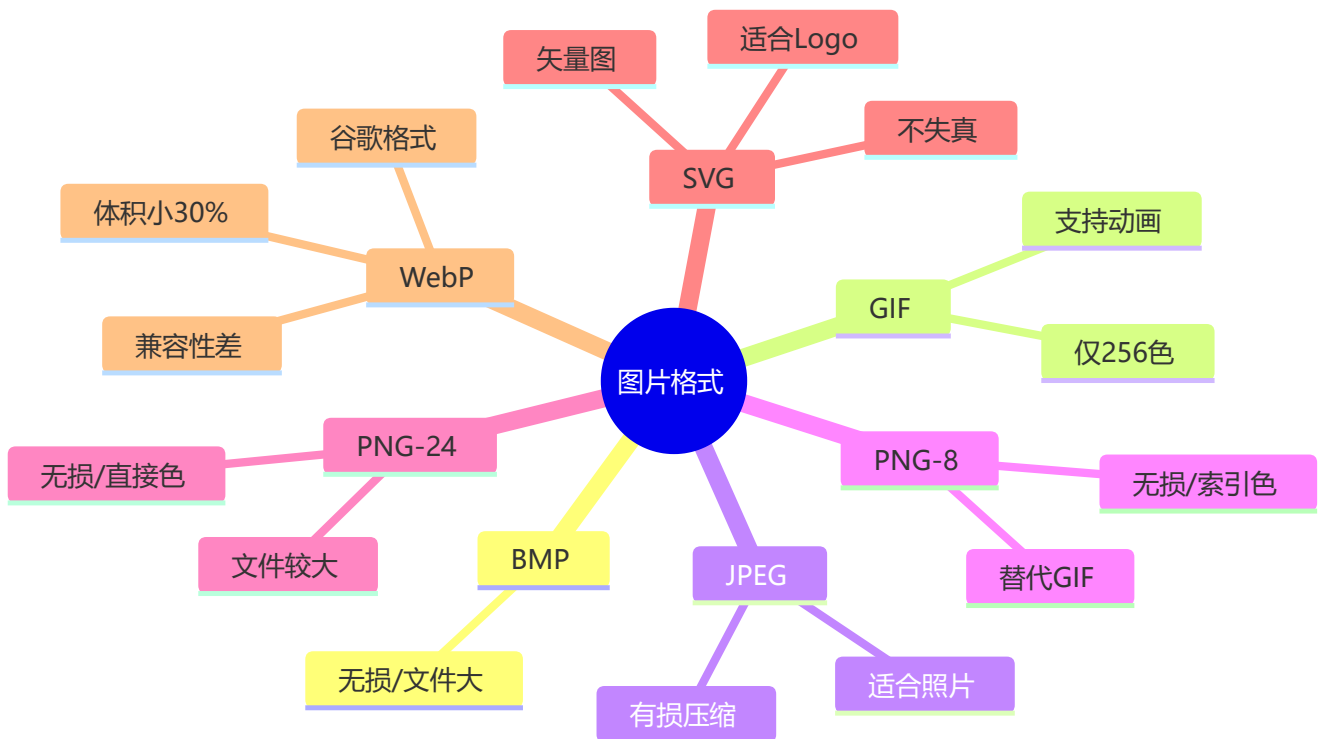
1 如何对项目中的图片进行优化？

💡 **要点：** 图片优化策略包括：CSS 代替图片、按需裁剪尺寸、Base64 小图内联、雪碧图合并图标、选择 WebP/AVIF 等现代格式。

1. 不用图片。很多时候会使用到很多修饰类图片，其实这类修饰图片完全可以用 CSS 去代替。
2. 对于移动端来说，屏幕宽度就那么点，完全没有必要去加载原图浪费带宽。可以计算出适配屏幕的宽度，然后去请求相应裁剪好的图片。
3. 小图使用 base64 格式
4. 将多个图标文件整合到一张图片中（雪碧图）
5. 选择正确的图片格式：
 - 对于能够显示 WebP 格式的浏览器尽量使用 WebP 格式
 - 小图使用 PNG，其实对于大部分图标这类图片，完全可以使用 SVG 代替
 - 照片使用 JPEG

2 常见的图片格式及使用场景

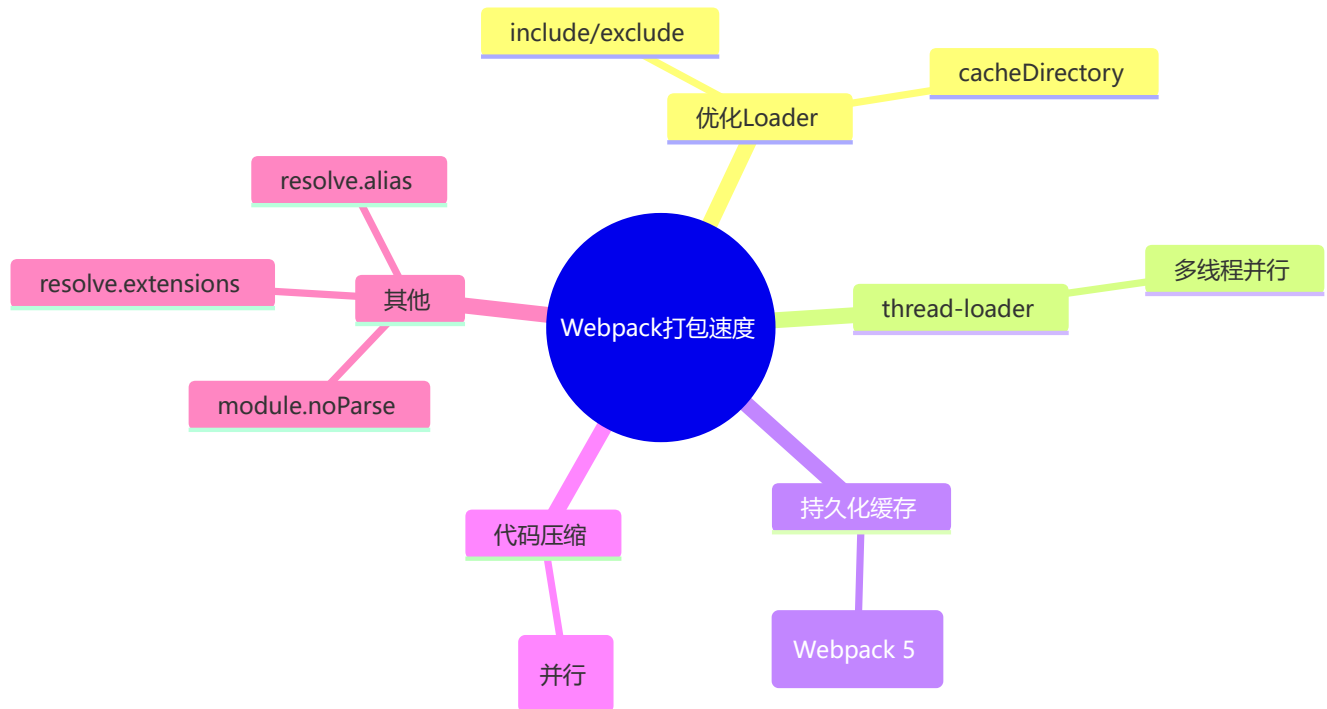
💡 **要点：** JPEG 适合照片（有损压缩），PNG 适合小图/透明（无损），WebP 比 JPEG 小 25-35%，SVG 矢量不失真适合 Logo，AVIF 压缩率最高但兼容性有限。



六、Webpack优化

1 如何提高 Webpack 的打包速度？

💡 要点：通过优化 Loader (include/exclude/cacheDirectory) 、thread-loader 多线程打包、Webpack 5 持久化缓存、terser-webpack-plugin 并行压缩等方式提升打包速度。



◆ (1) 优化 Loader

```
module.exports = {
  module: {
    rules: [
      {
        test: /\.js$/,
        loader: 'babel-loader',
        include: [resolve('src')],
        exclude: /node_modules/
      }
    ]
  }
}
```

```
loader: 'babel-loader?cacheDirectory=true'
```

◆ (2) thread-loader 多线程并行

```
module.exports = {
  module: {
    rules: [
      {
```

```

    test: /\.js$/,
    include: [resolve('src')],
    exclude: /node_modules/,
    use: [
      { loader: 'thread-loader', options: { workers: 4 } },
      'babel-loader?cacheDirectory'
    ]
  }
]
}
}
}

```

◆ (3) Webpack 5 持久化缓存

```

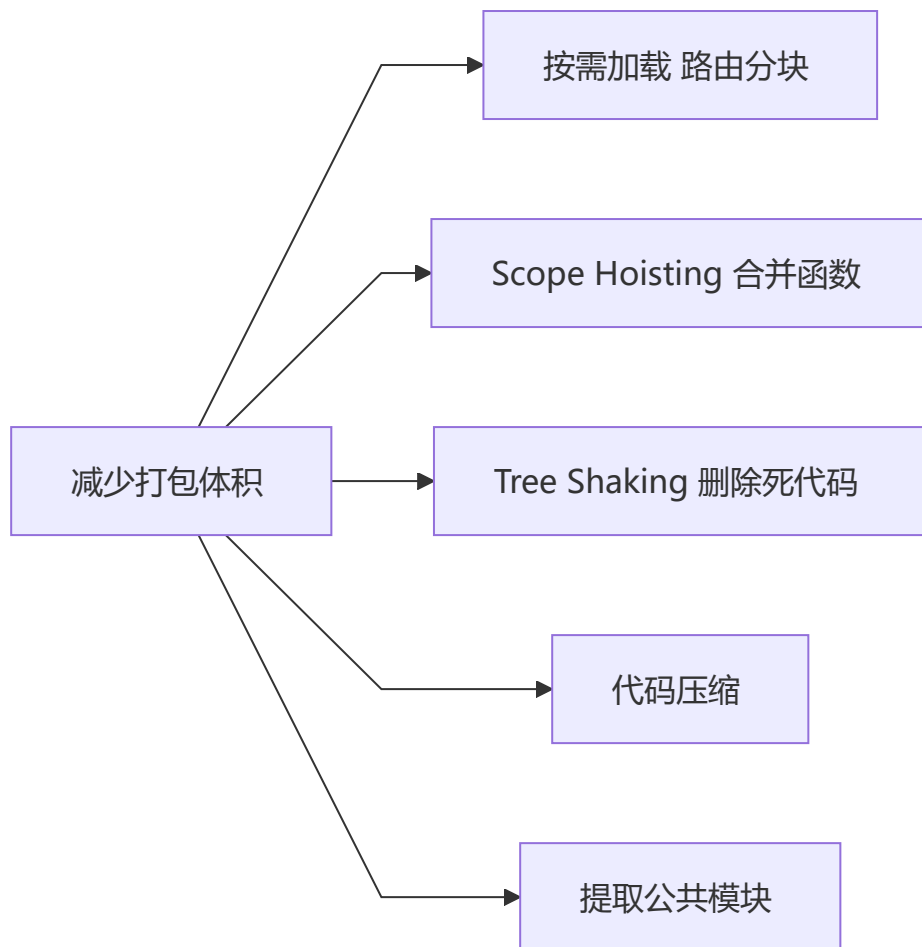
// webpack.config.js
module.exports = {
  cache: {
    type: 'filesystem',
    buildDependencies: {
      config: [__filename]
    }
  }
}
}

```

💡 **替代说明：**Webpack 5 的持久化缓存（`cache: { type: 'filesystem' }`）可完全替代 DllPlugin。首次构建后缓存中间产物到磁盘，后续仅重新编译变更模块，无需额外维护 dll 打包配置。

2 如何减少 Webpack 打包体积

💡 **要点：**通过按需加载（路由分块/chunk 分割）、Scope Hoisting（函数合并减少包裹代码）、Tree Shaking（删除未引用代码）等方式显著减少打包体积。



◆ (1) 按需加载

将每个路由页面单独打包为一个文件，当使用的时候再去下载对应文件。

◆ (2) Scope Hoisting

Scope Hoisting 会分析出模块之间的依赖关系，尽可能的把打包出来的模块合并到一个函数中去。

```
// 未使用 Scope Hoisting
[
  /* 0 */
  function (module, exports, require) { /*...*/ },
  /* 1 */
  function (module, exports, require) { /*...*/ }
]

// 使用 Scope Hoisting
[
  /* 0 */
  function (module, exports, require) { /*...*/ }
]
```

```
module.exports = {
  optimization: {
    concatenateModules: true
  }
}
```

◆ (3) Tree Shaking

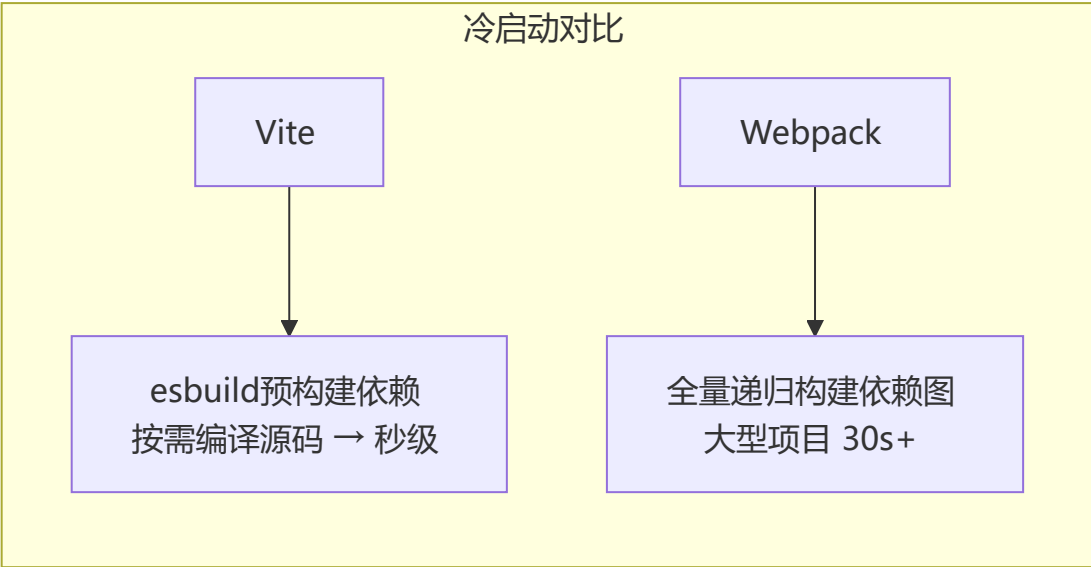
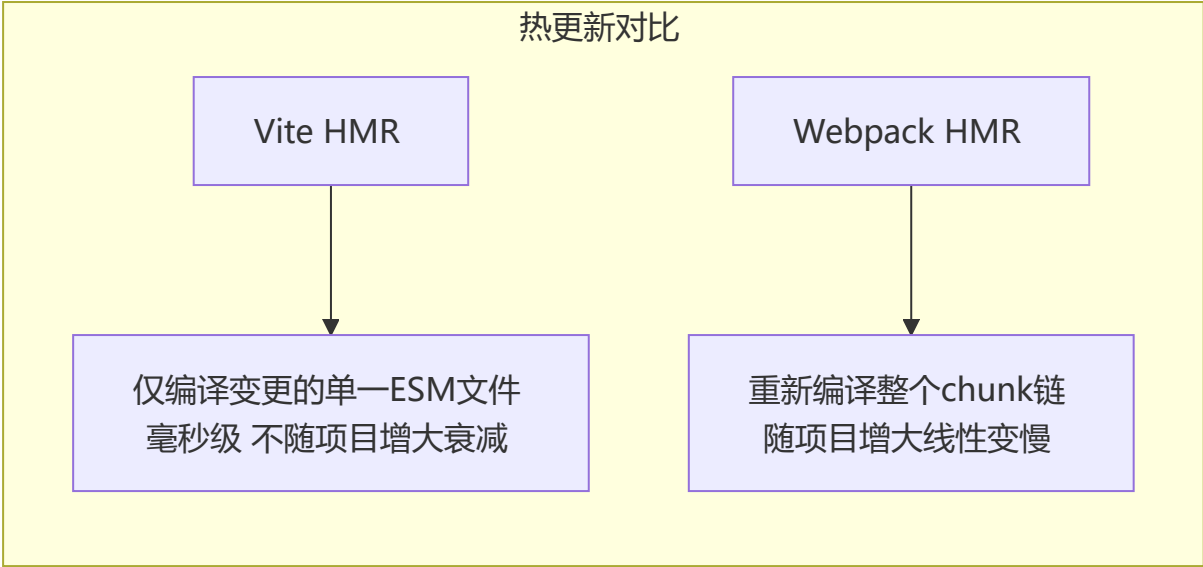
Tree Shaking 可以实现删除项目中未被引用的代码，开启生产环境就会自动启动这个优化功能。

```
// webpack.config.js
module.exports = {
  ...
  optimization: {
    usedExports: true
  }
}
```

⚠ **注意：**Tree Shaking 依赖 ES Module (`import/export`) 的静态分析特性，CommonJS (`require()`) 无法进行 Tree Shaking。确保项目中三方库也使用 ES Module 版本（如 `lodash-es` 替代 `lodash`）。

4 Vite vs Webpack 性能对比

💡 **要点：**Vite 基于 ESM 开发服务器 + esbuild 预构建，冷启动秒级；Webpack 全量构建依赖图，大型项目启动慢。Vite HMR 仅编译变更文件，Webpack 需重新打包整个 chunk。



对比维度	Vite	Webpack
冷启动	esbuild 预构建 + 按需编译，秒级	全量构建依赖图，大型项目 30-60s
HMR 速度	仅编译变更的 ESM 模块，毫秒级	重新打包整个 chunk，项目越大越慢
生产构建	Rollup (Tree-shaking 优秀，输出更小)	TerserPlugin + 兼容性更好
代码分割	Rollup manualChunks 配置	SplitChunksPlugin 灵活度更高
开发服务器	原生 ESM，无需打包	需要 webpack-dev-server + 内存打包
缓存策略	强缓存预构建依赖 + 源码协商缓存	持久化缓存 + loader 缓存
生态	成熟，主流框架官方推荐	最丰富，大量 Loader/Plugin
多页应用	通过 rollupOptions.input 配置	灵活多入口配置
SSR 支持	内置 SSR 构建 (vite build --ssr)	需额外配置 (webpack-node-externals)

何时选 Vite:

- 新项目开发，追求极致的 HMR 体验
- 浏览器兼容目标为现代浏览器 (ES2020+)
- 使用 Vue/React/Svelte 等主流框架 (官方推荐)
- 对构建速度敏感的大型项目

何时选 Webpack:

- 需要兼容旧浏览器 (ES5)
- 使用较老或小众的 Loader/Plugin
- 复杂的 Module Federation 微前端架构
- 需要极大量的自定义构建配置

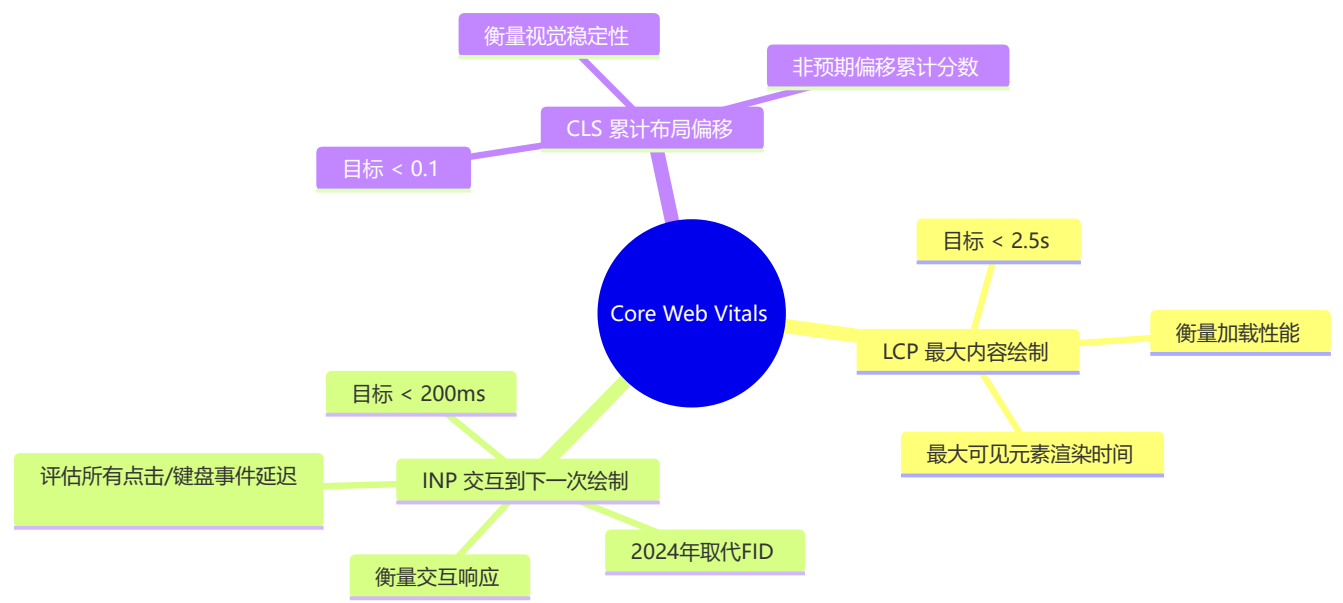
💡 **Tip:** 2024-2026 年主流趋势已转向 Vite。Webpack 5 的项目建议渐进迁移，新项目直接选择 Vite。对于需要 Webpack 生态但追求速度的团队，可考虑 **Rspack** (Rust 实现，兼容 Webpack API)。

七、Core Web Vitals

1 核心指标概览

💡 **要点:** Core Web Vitals 三大核心指标：LCP (最大内容绘制 < 2.5s) 衡量加载性能、INP (交互响应 < 200ms, 2024 年取代 FID) 衡量交互性能、CLS (累计布局偏移 < 0.1) 衡量视觉稳定性。

Core Web Vitals 是 Google 制定的用户体验量化标准，2024年后核心指标为 **LCP**、**INP**、**CLS** 三项。

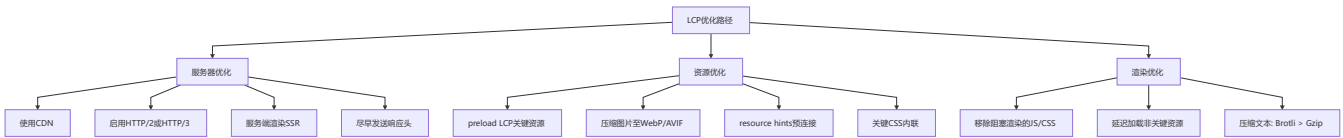


2 LCP 优化

💡 **要点:** LCP 优化路径包括服务器优化 (CDN/HTTP2/SSR)、资源优化 (preload 关键资源/图片压缩)、渲染优化 (消除阻塞/延迟非关键资源)。

LCP (Largest Contentful Paint) 衡量页面主要内容对用户可见的时间。常见 LCP 元素包括 img、video、含背景图的 div、大型文本块等。

优化目标: < 2.5s



```
<!-- preload LCP 图片 -->
<link rel="preload" as="image" href="hero.webp">

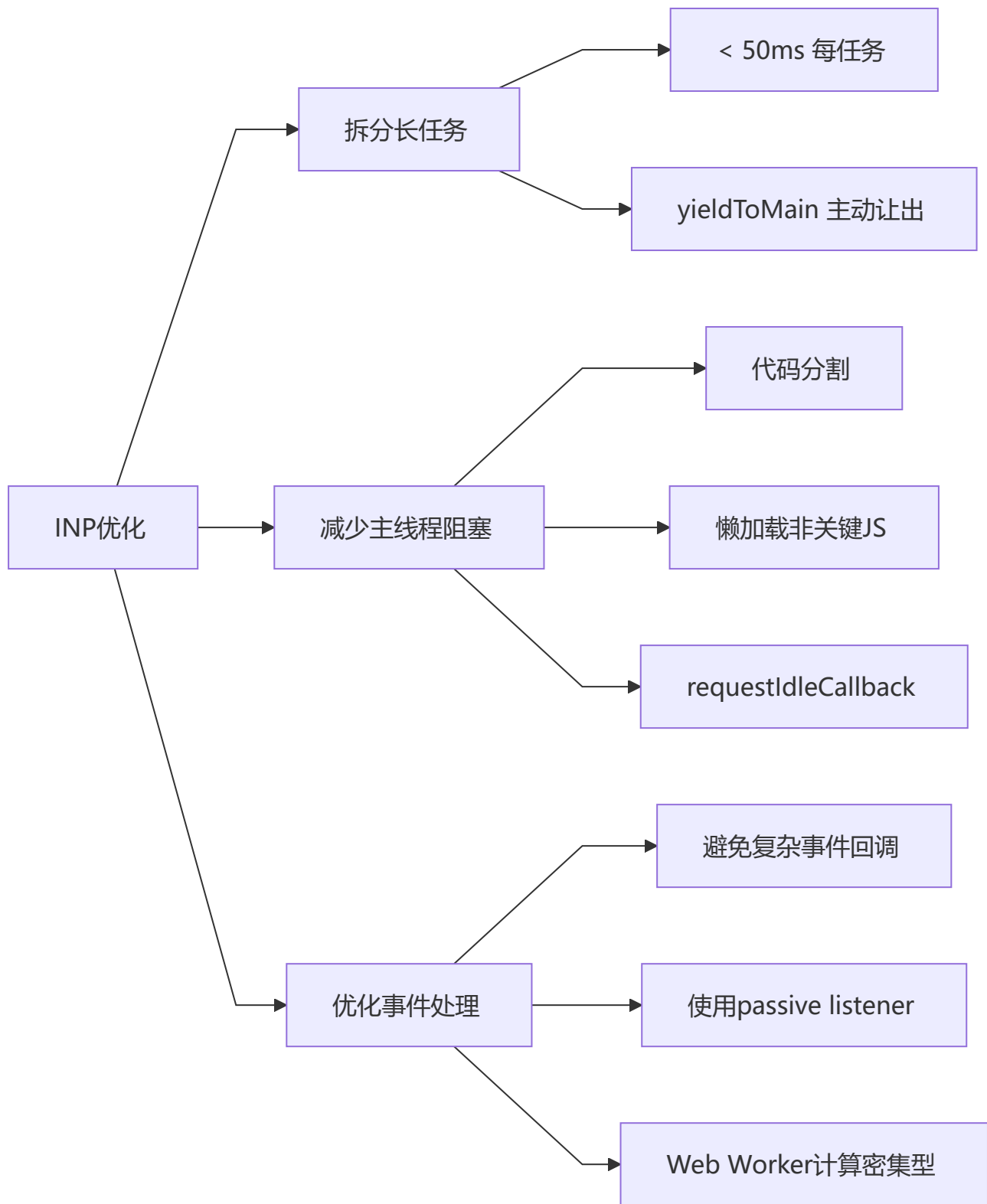
<!-- preconnect 第三方源 -->
<link rel="preconnect" href="https://fonts.googleapis.com">
```

3 INP 优化

💡 要点: INP 优化核心是拆分长任务 (< 50ms 每任务)、减少主线程阻塞 (代码分割/懒加载)、优化事件处理 (passive listener/Web Worker) 。

INP (Interaction to Next Paint) 评估页面所有交互操作的响应延迟, 取最差或接近最差的延迟作为评分依据。2024 年3月正式取代 FID。

优化目标: < 200ms



 **说明：** `yieldToMain()` 通过 `setTimeout(0)` 主动让出主线程，让浏览器有机会处理用户交互。每处理 50 项数据让出一次，避免长时间阻塞主线程。

```
// yieldToMain - 主动让出主线程
function yieldToMain() {
  return new Promise(resolve => {
    setTimeout(resolve, 0);
  });
}

async function processLargeData(items) {
  for (let i = 0; i < items.length; i++) {
    processItem(items[i]);
    if (i % 50 === 0) await yieldToMain();
  }
}
```

```
// requestIdleCallback 利用空闲时间
requestIdleCallback((deadline) => {
  while (deadline.timeRemaining() > 0 && tasks.length > 0) {
    processTask(tasks.shift());
  }
}, { timeout: 2000 });
```

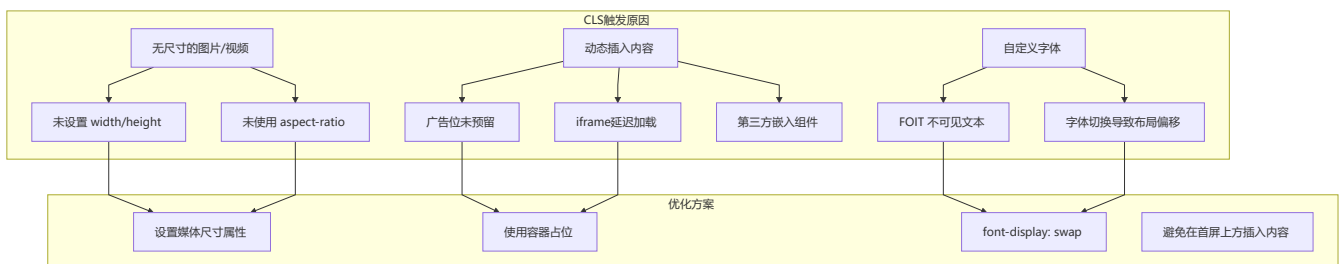
4 CLS 优化

要点：CLS 优化需为媒体元素设置 width/height + aspect-ratio、为广告位预留空间、使用 font-display: swap + size-adjust 减少字体切换偏移、避免首屏上方动态插入内容。

CLS (Cumulative Layout Shift) 量化页面在加载过程中内容的**非预期偏移**。偏移分数 = 影响比例 × 移动距离。

优化目标： < 0.1

CLS 触发原因分析：



```
/* 设置尺寸属性 */
img, video {
  width: 100%;
  height: auto;
  aspect-ratio: 16 / 9;
}

/* 为广告位预留空间 */
.ad-slot {
  min-height: 250px;
  width: 100%;
}
```

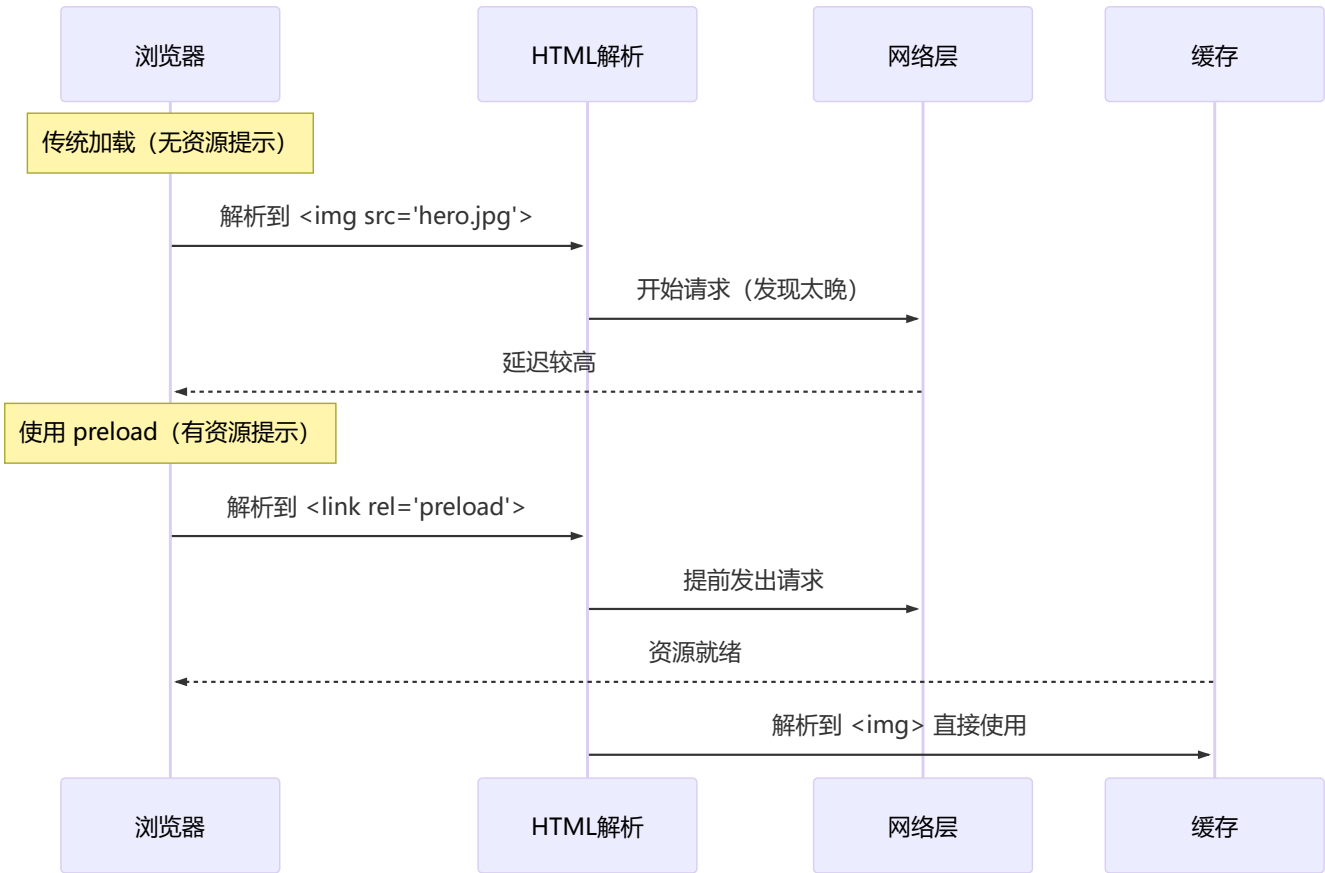
```
/* 字体优化 */
@font-face {
  font-family: 'CustomFont';
  src: url('/fonts/CustomFont.woff2') format('woff2');
  font-display: swap;
}
```

🚀 八、资源加载优化

1 资源提示 (Resource Hints)

💡 **要点：** Resource Hints 包括 preload（当前页关键资源）、prefetch（预取下一页）、preconnect（预建立连接）、dns-prefetch（DNS 预解析）、modulepreload（预加载 ES Module），是零成本的预加载手段。

资源提示让浏览器**提前发现并优先处理**关键资源，是零成本的预加载优化手段。



提示类型	用途	示例
preload	提前加载当前页面关键资源	<code><link rel="preload" as="font"></code>
prefetch	预取下一页可能用到的资源	<code><link rel="prefetch" href="/next-page.js"></code>
preconnect	提前建立源站连接 (DNS+TCP+TLS)	<code><link rel="preconnect" href="https://api.example.com"></code>

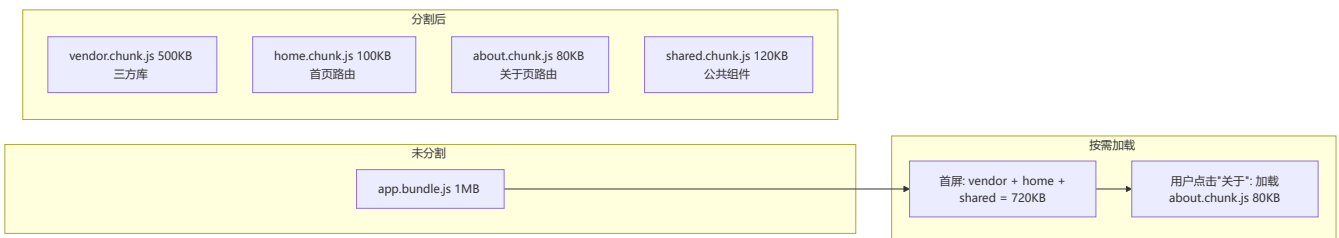
提示类型	用途	示例
<code>dns-prefetch</code>	仅提前解析DNS	<code><link rel="dns-prefetch" href="//example.com"></code>
<code>modulepreload</code>	预加载ES Module及其依赖	<code><link rel="modulepreload" href="/app.mjs"></code>

```
<!-- 资源提示示例 -->
<link rel="preload" href="/fonts/Inter.woff2" as="font" crossorigin>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="dns-prefetch" href="//www.googletagmanager.com">
<link rel="prefetch" href="/blog-page.js" as="script">
<link rel="modulepreload" href="/src/app.js">
```

2 代码分割策略

💡 **要点：** 代码分割将应用拆分为小 chunk 按需加载，React 用 `lazy()` + `Suspense`、Vue 用 `defineAsyncComponent`，Vite/Rollup 通过 `manualChunks` 手动拆分。

代码分割将应用拆分为更小的 chunk，按需加载，显著减少首屏体积。



📖 **说明：** React 通过 `lazy()` + `Suspense` 实现路由级代码分割，Vue 3 通过 `defineAsyncComponent` 实现组件级分割，两者都基于动态 `import()`。

```
// React: React.lazy 路由级别分割
const Home = React.lazy(() => import('./pages/Home'));
const About = React.lazy(() => import('./pages/About'));

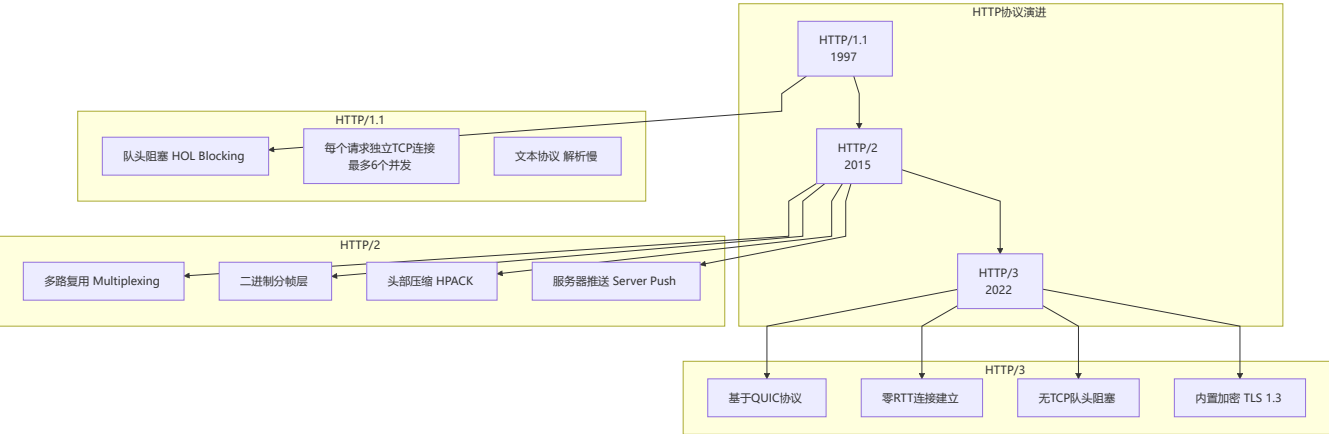
// Vue: defineAsyncComponent 组件级别分割
const HeavyChart = defineAsyncComponent(() =>
  import('./components/HeavyChart.vue')
);
```

📖 **说明：** Vite 通过 `manualChunks` 手动配置代码分割规则，将三方库和工具函数分别打包为独立 chunk，利用浏览器缓存减少重复加载。

```
// vite 自动代码分割 (vite.config.js)
export default defineConfig({
  build: {
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ['react', 'react-dom'],
          utils: ['lodash-es', 'date-fns'],
        }
      }
    }
  }
});
```

3 HTTP/2 vs HTTP/3

💡 **要点:** HTTP/2 通过多路复用、二进制分帧、头部压缩解决 HTTP/1.1 队头阻塞；HTTP/3 基于 QUIC/UDP，实现 0-RTT 连接和真正的无队头阻塞。

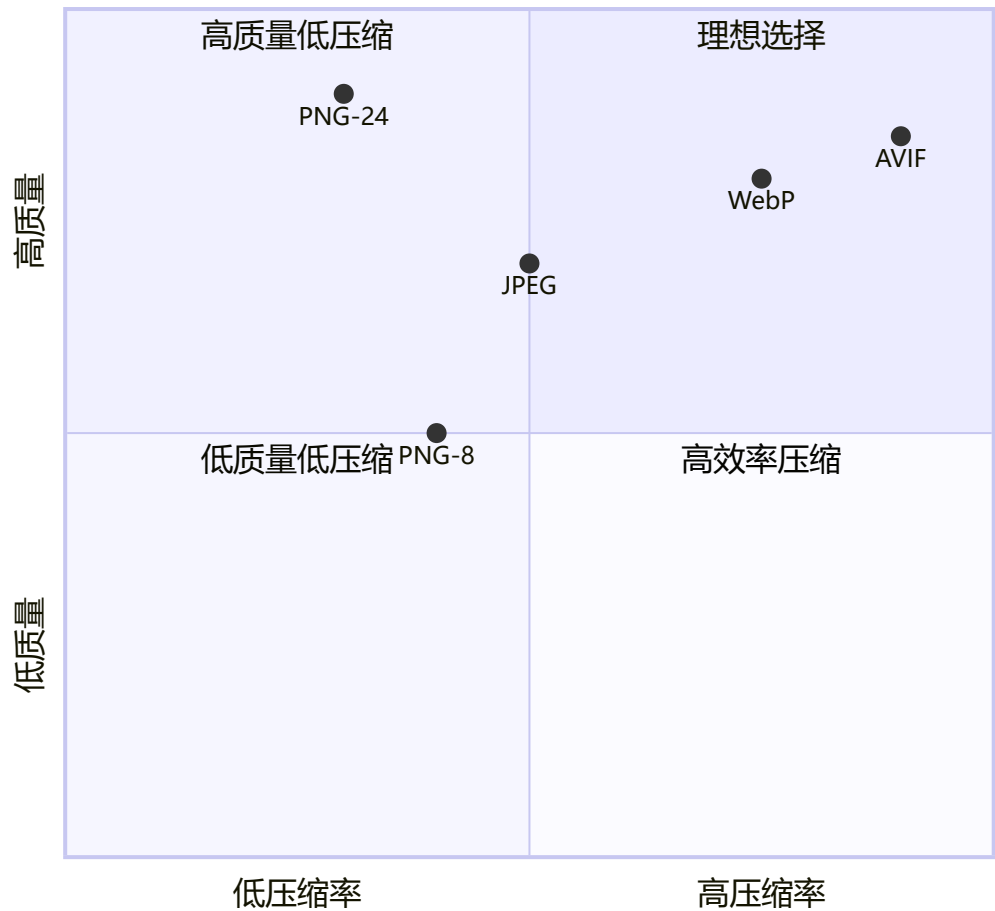


特性	HTTP/1.1	HTTP/2	HTTP/3
传输协议	TCP	TCP	QUIC (基于UDP)
多路复用	不支持	支持	支持
队头阻塞	应用层+传输层	传输层(TCP)	无
连接建立	3次握手	1次TCP + 协商	0-RTT / 1-RTT
头部压缩	无	HPACK	QPACK
服务器推送	无	支持	支持

4 压缩算法

💡 **要点:** Brotli 压缩率优于 Gzip (~80% vs ~70%) 但速度较慢；图片格式中 WebP 比 JPEG 小 25-35%，AVIF 比 JPEG 小 50%，JPEG XL 可小 60% 但兼容性最低。

图片格式对比



文本压缩	压缩比	速度	浏览器支持
Gzip	~70%	快	100%
Brotli (质量11)	~80%	较慢	~96%

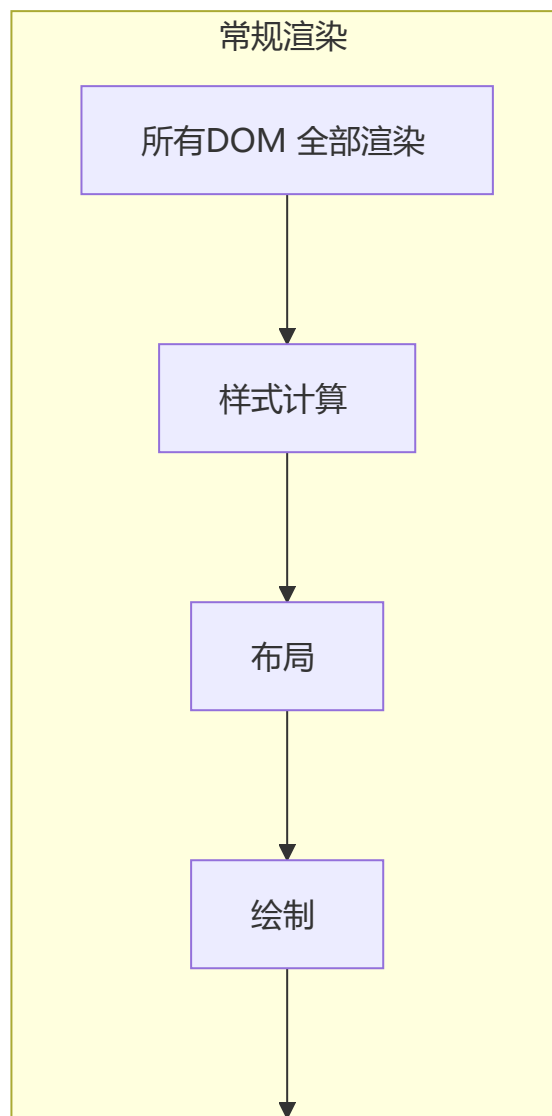
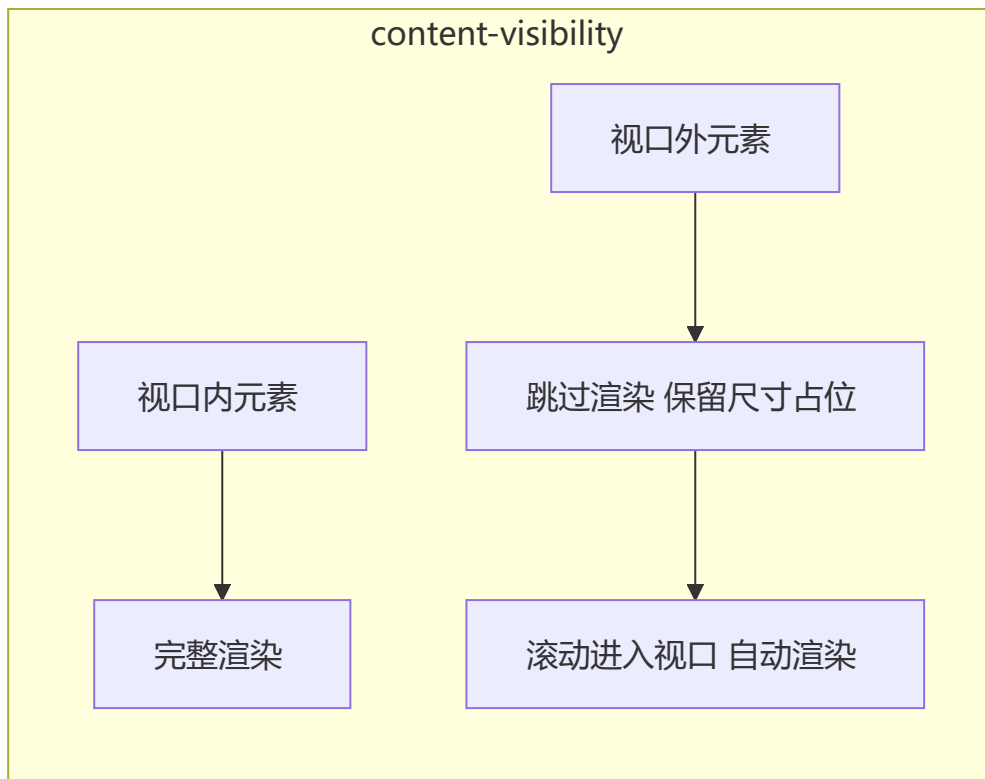
图片格式	压缩类型	相对JPEG体积	浏览器支持
JPEG	有损	基准	100%
WebP	有损/无损	小25-35%	~96%
AVIF	有损/无损	小50%	~92%
JPEG XL	有损/无损	小60%	~20%

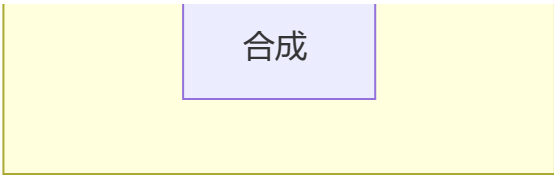
九、渲染性能优化

1 content-visibility: auto

💡 要点: `content-visibility: auto` 可跳过视口外元素的样式计算、布局和绘制, 配合 `contain-intrinsic-size` 使用, 一行 CSS 实现长列表虚拟渲染。

`content-visibility: auto` 是 CSS 原生虚拟渲染属性, 浏览器自动跳过屏幕外元素的渲染 (样式计算、布局、绘制)。





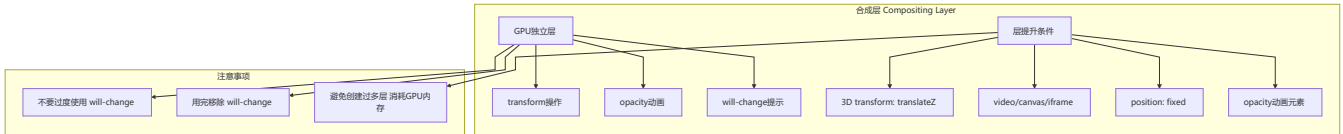
特性	content-visibility	传统懒加载
实现方式	CSS属性	JS监听滚动
渲染跳过	样式+布局+绘制	仅跳过资源加载
内存占用	低（跳过渲染）	中
兼容性	现代浏览器	全兼容
使用难度	一行CSS	需要JS逻辑

```
/* 长列表优化 - 一行CSS */
.long-list-item {
  content-visibility: auto;
  contain-intrinsic-size: 200px; /* 占位高度 */
}
```

2 will-change 和 GPU 加速

要点： will-change 通知浏览器提前优化该元素，CSS 3D transforms（如 `translateZ(0)`）可触发 GPU 合成层，但不要过度使用以免消耗 GPU 内存。

will-change 告知浏览器提前优化特定元素，而 CSS 3D transforms 可触发 GPU 合成层，将渲染从 CPU 转移到 GPU。



```
/* 开启GPU加速 */
.gpu-accelerated {
  transform: translateZ(0);
  /* 或 will-change: transform; */
}

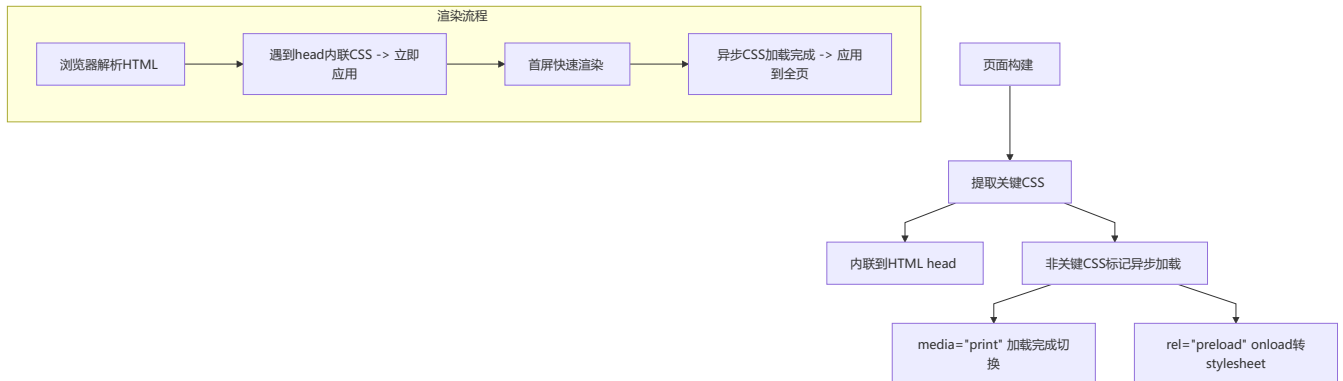
/* 动画开始前提示 */
.animated-element {
  will-change: transform, opacity;
}

/* 动画结束后移除 */
.animated-element.playing {
  will-change: auto;
}
```

3 Critical CSS

💡 **要点:** Critical CSS 将首屏关键样式内联到 `<head>`，非关键 CSS 通过 `preload + onload` 异步加载，可用 `critters-webpack-plugin` 自动提取内联。

Critical CSS 将首屏渲染必需的样式**内联**到 HTML `<head>` 中，非关键样式异步加载，消除 CSS 阻塞渲染。



```
<!-- 关键CSS内联 -->
<head>
  <style>
    /* 首屏关键样式直接内联 */
    .header { display: flex; ... }
    .hero { height: 100vh; ... }
  </style>

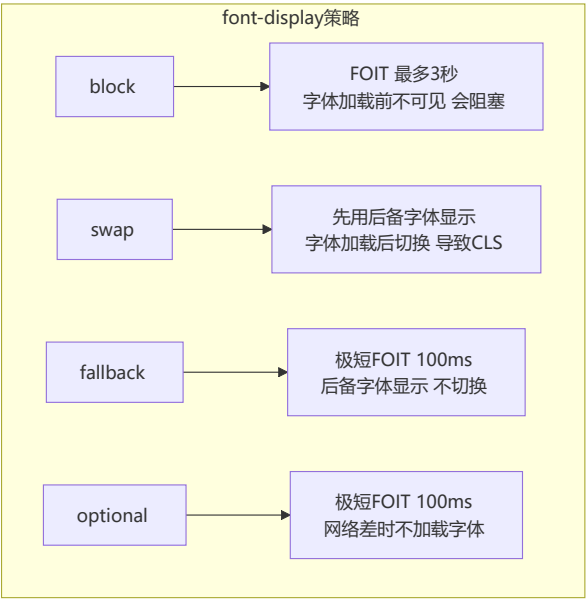
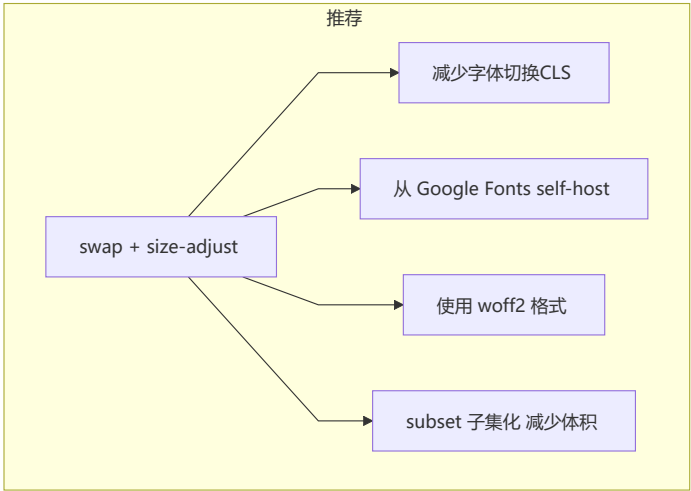
  <!-- 非关键CSS异步加载 -->
  <link rel="preload" href="/styles/full.css" as="style"
        onload="this.onload=null;this.rel='stylesheet'">
  <noscript><link rel="stylesheet" href="/styles/full.css"></noscript>
</head>
```

```
# 使用 critters (webpack插件) 自动提取内联
npm i critters-webpack-plugin
```

4 字体优化

💡 **要点:** `font-display: swap` 防止字体阻塞文本渲染，配合 `size-adjust` 减少字体切换引起的 CLS；使用 `woff2` 和子集化 (subset) 可大幅减小字体体积。

自定义字体加载策略直接影响 **CLS** 和首屏渲染速度。



```
/* font-display 策略 */
@font-face {
  font-family: 'Inter';
  src: url('/fonts/Inter.woff2') format('woff2');
  font-display: swap;
  font-weight: 400;
}

/* size-adjust 减少CLS */
@font-face {
  font-family: 'Inter Fallback';
  src: local('Arial');
  size-adjust: 98%;
  ascent-override: 90%;
  descent-override: 22%;
}
```

策略	FOIT时间	行为	适用场景
block	最多3s	不可见期间不显示文本	Logo、品牌字体
swap	0ms	立即用后备字体	正文内容
fallback	100ms	短FOIT后退化为后备	一般推荐
optional	100ms	网络差时不加载	装饰性字体

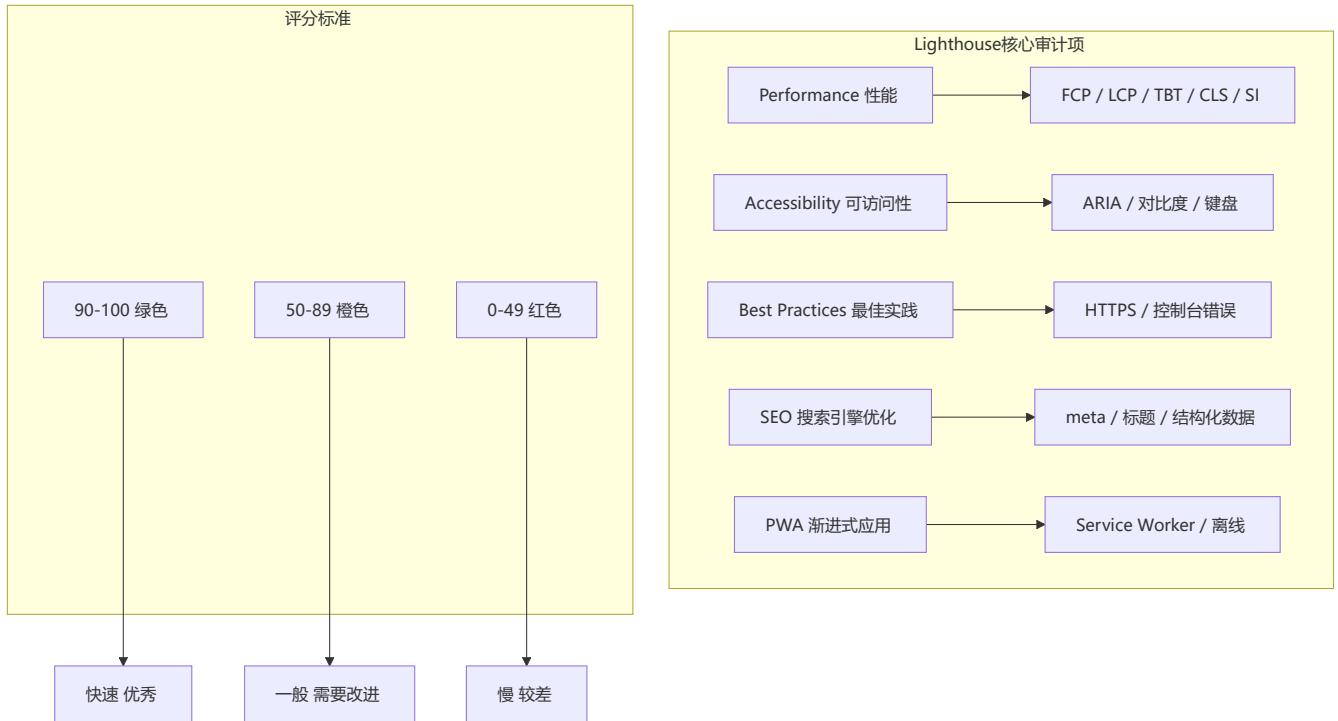


十、监测与分析

1 Lighthouse

💡 **要点：** Lighthouse 审计性能（FCP/LCP/TBT/CLS/SI）、可访问性、SEO、最佳实践、PWA 五个维度，评分 90+ 为绿色优秀，50-89 橙色需改进，0-49 红色较差。

Lighthouse 是 Google 开源的自动化网站审计工具，覆盖性能、可访问性、SEO 等多个维度。

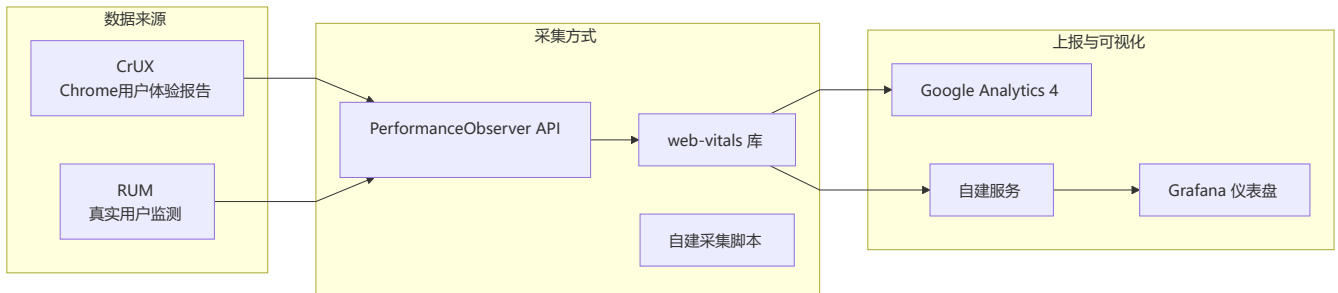


```
# Lighthouse CI - 自动化
npx lhci autorun --collect.url=https://example.com
```

2 Web Vitals 监测

💡 **要点：** Web Vitals 监测需结合 CrUX（实验室数据）和 RUM（真实用户数据），通过 PerformanceObserver API 或 web-vitals 库采集，上报到 GA4 或自建服务。

全面监测需要结合 **CrUX（实验室数据）** 和 **RUM（真实用户数据）**。



📄 **说明：** PerformanceObserver API 可以直接监听 LCP、CLS、INP 等核心指标事件，是采集 Web Vitals 的基础 API。

```
// Performance Observer API
const observer = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    console.log(entry.name, entry.startTime, entry.duration);
  }
});

observer.observe({ type: 'largest-contentful-paint', buffered: true });
observer.observe({ type: 'layout-shift', buffered: true });
observer.observe({ type: 'first-input', buffered: true });
```

 **说明：** `web-vitals` 库封装了 `PerformanceObserver`，提供更简洁的 API 来采集 Core Web Vitals 指标，支持回调函数上报数据。

```
// 使用 web-vitals 库
import { onLCP, onINP, onCLS } from 'web-vitals';

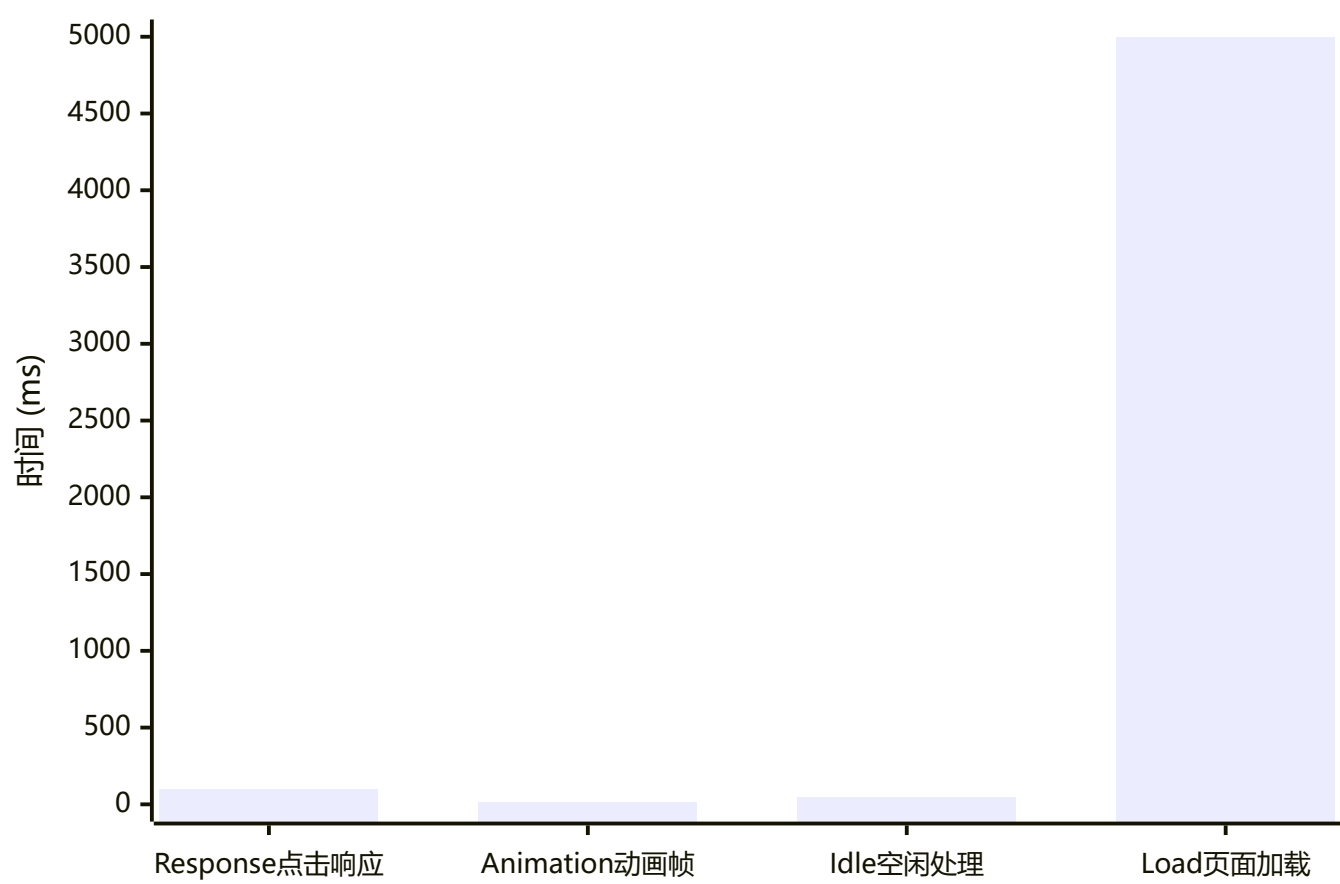
onLCP(console.log);
onINP(console.log);
onCLS(console.log);
```

3 RAIL 模型

 **要点：** RAIL 以用户为中心：Response 响应 < 100ms、Animation 动画帧 < 16ms（60fps）、Idle 利用空闲时间处理延迟任务、Load 加载 < 5s（3G 网络）。

RAIL 是 Google 提出的**以用户为中心**的性能模型，定义了四个关键阶段的响应目标。

RAIL 性能目标



阶段	目标	说明
Response 响应	< 100ms	用户操作在100ms内得到反馈
Animation 动画	< 16ms (60fps)	每帧在16ms内完成渲染
Idle 空闲	利用空闲时间	用 requestIdleCallback 处理延迟任务
Load 加载	< 5s	首屏内容在5秒内完成（3G网络）

 **说明：**RAIL 的 Response 阶段要求用户在 100ms 内得到反馈，因此耗时任务需异步处理；Animation 阶段需保证每帧在 16ms 内完成渲染以保持 60fps。

```
// RAIL Response - 事件处理
button.addEventListener('click', () => {
  // 必须在100ms内给出反馈
  showLoadingIndicator(); // 立即反馈

  // 耗时任务异步处理
  setTimeout(() => {
    processHeavyTask(); // 后续处理
  }, 50);
});

// RAIL Animation - 保持60fps
function animate() {
```

```
const start = performance.now();

updateAnimation(); // 必须在16ms内完成

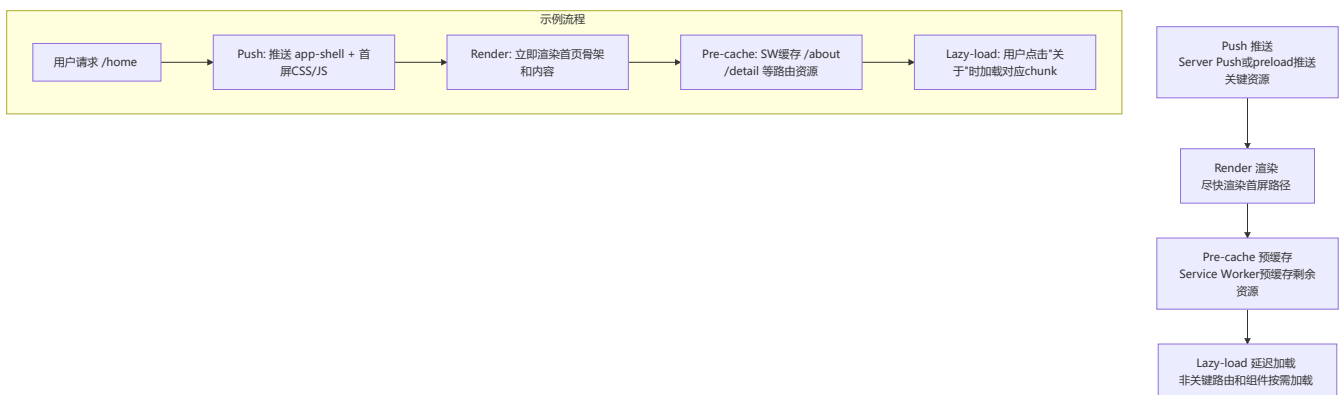
const elapsed = performance.now() - start;
if (elapsed > 16) {
  console.warn(`Frame dropped: ${elapsed}ms`);
}

requestAnimationFrame(animate);
}
```

4 PRPL 模式

💡 **要点：** PRPL = Push（推送关键资源）、Render（尽快渲染首屏）、Pre-cache（Service Worker 预缓存）、Lazy-load（按需加载非关键路由/组件）。

PRPL 是 Google 提出的 Web 应用性能模式，专为移动端和现代浏览器设计。



📖 **说明：** Workbox 是 Google 推出的 Service Worker 工具库，`precacheAndRoute` 自动预缓存构建资源，`registerRoute` + `NetworkFirst` 策略实现文档页面的网络优先缓存。

```
// service worker 预缓存 (workbox)
import { precacheAndRoute } from 'workbox-precaching';
precacheAndRoute(self.__WB_MANIFEST); // 自动预构建资源

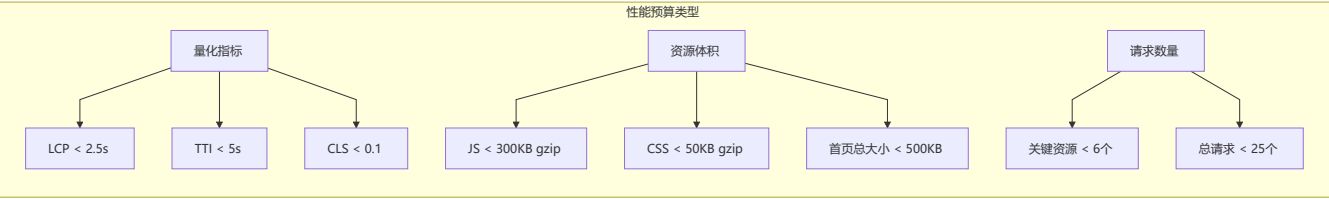
// 运行时缓存
import { registerRoute } from 'workbox-routing';
import { NetworkFirst } from 'workbox-strategies';

registerRoute(
  ({ request }) => request.destination === 'document',
  new NetworkFirst()
);
```

5 性能预算

💡 **要点：**性能预算设定量化指标（LCP/TTI/CLS）、资源体积（JS < 300KB gzip）和请求数量上限，通过 `bundleSize` 或 `webpack-bundle-analyzer` 在 CI 中自动检查。

性能预算（Performance Budget）是团队约定的**性能指标上限**，防止性能退化。



```
# 使用 bundleSize 设置体积预算
npx bundleSize --files "dist/**/*.js" --max-size "300 kB"

# webpack-bundle-analyzer 可视化分析
npx webpack-bundle-analyzer dist/stats.json
```

```
// bundleSize 配置 (package.json)
{
  "bundleSize": [
    { "path": "./dist/vendor.*.js", "maxSize": "250 kB" },
    { "path": "./dist/main.*.js", "maxSize": "50 kB" },
    { "path": "./dist/*.css", "maxSize": "30 kB" }
  ]
}
```

🌟 十一、前沿性能技术

1 Edge Computing

💡 **要点：**边缘计算将计算能力下沉到 CDN 节点，Edge Functions（如 Cloudflare Workers、Vercel Edge Functions）可在边缘运行轻量计算，极大降低响应延迟。

边缘计算将计算能力从集中式服务器**下沉到CDN节点**，极大降低用户到服务器的物理距离和响应时间。

技术	说明	代表产品
CDN	静态资源缓存分发	Cloudflare / Akamai
Edge Functions	在CDN节点运行轻量计算	Vercel Edge Functions
边缘SSR	在边缘节点完成服务器端渲染	Cloudflare Workers + SSR
Edge Cache	动态内容在边缘缓存	Fly.io / Deno Deploy

📖 **说明：**Cloudflare Workers 在边缘节点运行 JavaScript，可根据请求路径动态渲染 HTML 或直接回源获取静态资源，实现零服务器延迟。

```
// Cloudflare workers - 边缘响应
export default {
  async fetch(request) {
    const url = new URL(request.url);

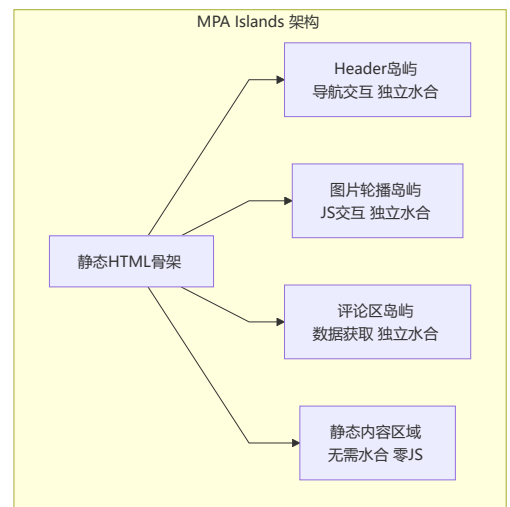
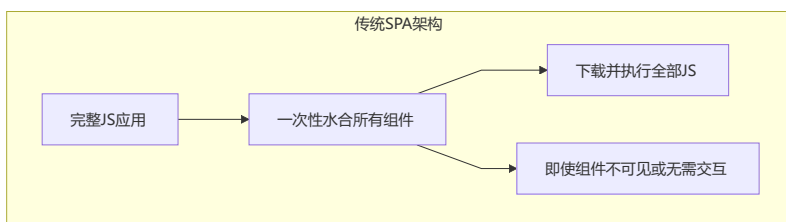
    // 边缘HTML渲染
    if (url.pathname === '/product') {
      const product = await getProductFromKV(url.searchParams.get('id'));
      return new Response(renderHTML(product), {
        headers: { 'Content-Type': 'text/html' }
      });
    }

    // 静态资源直接返回
    return fetch(request);
  }
}
```

2 Islands Architecture (岛屿架构)

💡 **要点：** 岛屿架构将交互组件视为独立"岛屿"分别水合，静态 HTML 无需 JS，代表框架为 Astro，通过 `client:load` / `client:visible` 指令控制岛屿加载时机。

岛屿架构将页面中的**交互组件**视为独立的"岛屿"，每个岛屿可以独立水合，而非一次性水合整个页面。



📖 **说明：** Astro 的岛屿架构将交互组件标记为独立"岛屿"，`client:load` 表示页面加载时立即水合，`client:visible` 表示进入视口时水合，其余静态部分零JS。

```
// Astro 岛屿组件
---
// 静态部分 - 构建时渲染
const posts = await fetchPosts();
---
<html>
  <body>
    <!-- 静态HTML - 零JS -->
    <header>
      <nav>{posts.map(p => <a href={p.url}>{p.title}</a>)}</nav>
```

```
</header>

<!-- 交互岛屿 - 仅此处加载JS -->
<ImageCarousel client:load />

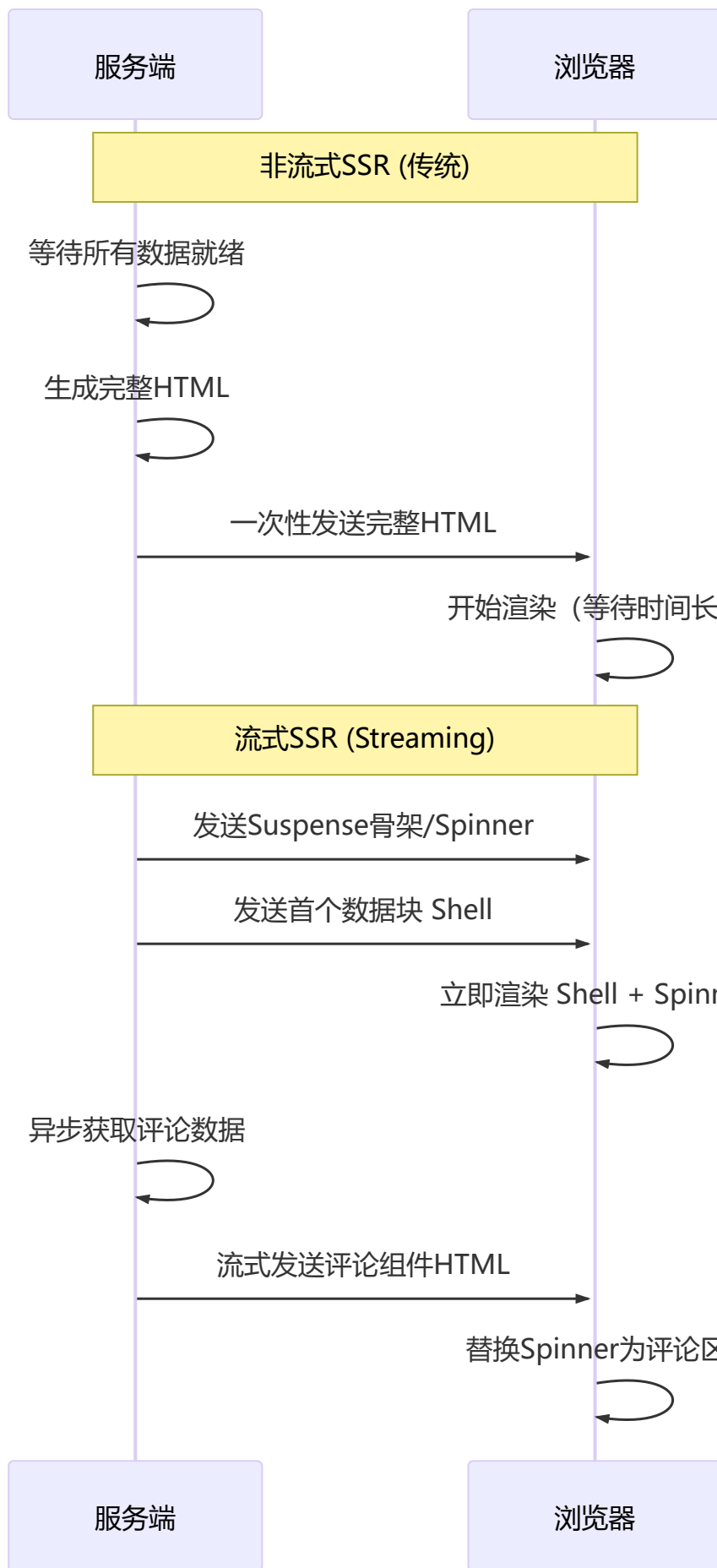
<!-- 首屏不可见岛屿 - 进入视口加载 -->
<CommentSection client:visible />

<!-- 静态内容 -->
<article>{content}</article>
</body>
</html>
```

3 Streaming SSR

💡 **要点：**流式 SSR 将 HTML 分块流式发送，结合 React Suspense 让浏览器尽早渲染 Shell 骨架，无需等待完整 HTML 生成，显著提升首屏感知速度。

流式 SSR 将服务器端渲染的 HTML **分块发送** 到客户端，让浏览器可以尽早开始渲染，无需等待完整 HTML 生成。



 **说明：** React 18 的 `renderToPipeableStream` 支持流式 SSR，`Suspense` 包裹的组件可独立流式发送，浏览器收到 Shell 后立即渲染无需等待所有数据。

```
// React 18 Streaming SSR
import { Suspense } from 'react';
import { renderToPipeableStream } from 'react-dom/server';

function handleRequest(req, res) {
  const { pipe } = renderToPipeableStream(
    <html>
      <body>
        <Header />
        <Suspense fallback=<Spinner />>
          <SlowComments /> { /* 流式发送 */ }
        </Suspense>
      </body>
    </html>
  );
  pipe(res); // 立即开始发送
}
```

4 Back/Forward Cache (bfcache)

 **要点：** bfcache 将整个页面（含 JS 堆）冻结到内存，前进/后退时瞬时恢复无需网络请求；需避免使用 `unload` 事件和 `Cache-Control: no-store` 阻止 bfcache。

bfcache 是浏览器在用户**前进/后退**导航时，将整个页面（包括JS堆）完整保存到内存中的机制，实现瞬时加载。



可能导致bfcache被排除的常见原因：

```
// ❌ 使用 unload 事件 - 将阻止bfcache
window.addEventListener('unload', () => {
  sendBeacon('/analytics');
});

// ✅ 使用 pagehide 替代
window.addEventListener('pagehide', () => {
  sendBeacon('/analytics');
});

// 检测是否从bfcache恢复
window.addEventListener('pageshow', (event) => {
  if (event.persisted) {
    console.log('页面从bfcache恢复');
    // 重新连接WebSocket、刷新过期数据等
  }
});
```

阻止bfcache的常见因素	解决方案
<code>unload</code> 事件	使用 <code>pagehide</code> 替代
<code>beforeunload</code> 事件	非必要不使用
<code>Cache-Control: no-store</code>	改为 <code>no-cache</code> 或 <code>private</code>
使用 <code>window.opener</code>	添加 <code>rel="noopener"</code>
活跃的 WebSocket 连接	在 <code>pagehide</code> 中关闭
未清理的 IndexedDB 事务	确保事务完成或中止

扩展阅读：

- [web.dev / vitals](#) - Core Web Vitals 官方指南
- [Chrome DevTools Performance](#) - 性能分析工具
- [BundlePhobia](#) - 检查npm包大小
- [PageSpeed Insights](#) - Google在线性能检测



十二、2025/2026 性能优化新趋势

1 INP 优化实战（2024-2026 核心指标）

💡 要点： INP（Interaction to Next Paint）于 2024 年 3 月正式取代 FID 成为 Core Web Vitals 核心指标，要求 90% 的交互响应 < 200ms。优化核心：拆分长任务、减少主线程阻塞、优化事件处理。

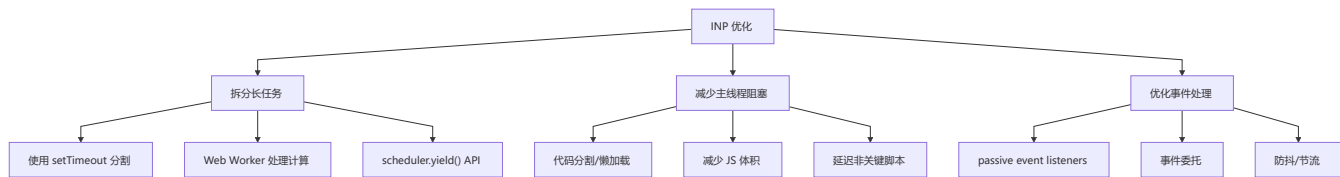
INP 测量方法：

```
// 使用 web-vitals 库
import { onINP } from 'web-vitals';

onINP((metric) => {
  console.log('INP:', metric.value);
  // 上报到分析平台
  sendToAnalytics('INP', metric.value);
});

// 原生 PerformanceObserver
const observer = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    console.log('Interaction:', entry.name, entry.startTime);
  }
});
observer.observe({ type: 'event', buffered: true });
```

INP 优化策略：



scheduler.yield() API (2025 新特性) :

```
// 旧方式: setTimeout 拆分任务
function processLargeArray(arr) {
  let i = 0;
  function chunk() {
    const end = Math.min(i + 100, arr.length);
    for (; i < end; i++) {
      processItem(arr[i]);
    }
    if (i < arr.length) {
      setTimeout(chunk, 0);
    }
  }
  chunk();
}

// 新方式: scheduler.yield() 让出主线程
async function processLargeArrayModern(arr) {
  for (let i = 0; i < arr.length; i++) {
    processItem(arr[i]);
    if (i % 100 === 0) {
      await scheduler.yield(); // 让出主线程，优先响应用户交互
    }
  }
}
```

2025 年 INP 优化清单:

- ✓ 所有交互任务拆分为 < 50ms 的块
- ✓ 使用 `scheduler.yield()` 替代 `setTimeout`
- ✓ 关键交互路径使用 `requestIdleCallback` 调度
- ✓ 事件监听器使用 `{ passive: true }`
- ✓ 避免同步 XHR 请求
- ✓ 使用 Web Worker 处理 CPU 密集型任务

2 2025/2026 性能工具链

工具	用途	2025/2026 更新
Lighthouse 12	性能审计	新增 INP 评分、AI 优化建议
web-vitals 4.0	指标采集	支持 INP、Navigation API 集成

工具	用途	2025/2026 更新
Chrome DevTools	性能分析	新增 INP 时间线、长任务火焰图
PageSpeed Insights	在线检测	真实用户数据 (CrUX) 集成
WebPageTest	深度分析	多地理位置、设备模拟
Calibre	持续监控	CI/CD 集成、性能预算

Lighthouse CI 集成：

```
# .github/workflows/lighthouse.yml
name: Lighthouse CI
on: push
jobs:
  lighthouse:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run build
      - uses: treosh/lighthouse-ci-action@v12
    with:
      uploadArtifacts: true
      budgetFile: ./lighthouse-budget.json
      assertMatrix: |
        [
          {
            "matching": ".*",
            "assertions": {
              "categories:performance": ["error", {"minScore": 0.9}],
              "interactive": ["error", {"maxNumericValue": 3000}]
            }
          }
        ]
```

3 边缘计算与性能优化

💡 要点：2025 年边缘计算成为性能优化新范式，Cloudflare Workers、Vercel Edge Functions、Deno Deploy 等平台将计算推向用户最近节点，TTFB 降低 50-80%。

边缘渲染 (Edge SSR)：

```
// Cloudflare Workers + React
import { renderToReadableStream } from 'react-dom/server';

export default {
  async fetch(request) {
    const stream = await renderToReadableStream(
      <App url={new URL(request.url)} />
    );
    return new Response(stream, {
      headers: { 'Content-Type': 'text/html' },
    });
  },
};
```

边缘缓存策略：

```
// Vercel Edge Middleware
import { NextRequest, NextResponse } from 'next/server';

export const config = {
  matcher: ['/api/:path*', '/products/:path*'],
};

export async function middleware(request: NextRequest) {
  const response = NextResponse.next();

  // 边缘缓存 1 小时
  response.headers.set('Cache-Control', 'public, max-age=3600, s-maxage=3600');

  // CDN 边缘节点缓存
  response.headers.set('CDN-Cache-Control', 'max-age=3600');

  return response;
}
```

边缘计算性能优势：

指标	传统 SSR	边缘 SSR	提升
TTFB	800-2000ms	100-400ms	50-80% ↓
全球延迟	高（单区域）	低（300+ 节点）	70% ↓
扩展性	需手动扩容	自动扩展	无限
成本	服务器固定成本	按请求计费	50% ↓

4

2026 性能优化前瞻

技术	描述	预期影响
AI 驱动优化	Lighthouse AI 自动分析性能瓶颈并给出修复建议	优化效率 3x ↑
Speculative Loading	浏览器自动预渲染用户可能访问的页面	LCP 30% ↓
WebTransport	替代 WebSocket，基于 QUIC 协议，延迟降低 50%	实时应用性能 ↑
Partial Hydration	仅激活可视区域组件，减少 JS 执行	TTI 40% ↓
Resumability (Qwik)	零 hydration 框架，按需恢复状态	首屏 JS 90% ↓
Server Components	React/Vue/Angular 全面支持服务端组件	客户端 JS 50% ↓

2026 性能优化最佳实践：

- 1. **优先优化 INP**：交互响应 < 200ms 成为硬性指标
- 2. **边缘优先**：SSR/SSG 部署到边缘节点
- 3. **AI 辅助**：使用 AI 工具自动分析和修复性能问题
- 4. **渐进式 hydration**：仅激活必要组件
- 5. **隐私优先**：在保护隐私前提下进行性能监控
- 6. **持续监控**：CI/CD 集成性能预算，防止回归

📌

导航

← 上一章：浏览器原理	🏠 返回主指南	→ 下一章：前端工程化
-----------------------------	-------------------------	-----------------------------